

1.队列

queue ()

队列 (queue)，是一种先进先出的数据结构

```
1  std::queue<T> q;  
2  q.push(x); // 元素 x 入队  
3  q.pop(); // 元素 x 出队  
4  q.top(); // 返回栈顶元素  
5  q.front(); // 返回队列的队首元素  
6  q.back(); // 返回队列的队尾元素  
7  q.empty(); // 检查是否为空,空返回1否则返回0  
8  q.size(); // 获取大小
```

deque ()

双端队列 (deque)，可以理解为一个双端数组

```
1  std::deque<int> dq;  
2  dq.push_back(x); // 在末尾加入一个元素  
3  dq.pop_back(); // 删除末尾的元素  
4  dq.push_front(x); // 在开头加入一个元素  
5  dq.pop_front(); // 删除开头的元素  
6  dq.front(); // 返回数组的开头元素  
7  dq.back() // 返回数组的末尾元素  
8  // 中括号随机访问, i 是下标  
9  dq[i] = x;  
10 dq.empty(); // 检查是否为空,空返回1否则返回0  
11 dq.size(); // 获取大小
```

priority_queue (<priority_queue>)

优先队列 (priority_queue)，也是一种堆，可以方便获取最值。

```
1  std::priority_queue<int> pq; // C++默认大根堆 栈顶元素永远最大 d  
2  std::priority_queue<int, std::vector<int>, std::greater<int>> pq; // 小根堆  
   栈顶元素永远最小 单调递减  
3  pq.push(x); // 将元素 x 放入堆中  
4  pq.pop(); // 将堆顶元素删除  
5  pq.top(); // 返回堆顶元素，大根堆返回最大值，小根堆返回最小  
6  pq.empty(); // 检查是否为空,空返回1否则返回0  
7  pq.size(); // 获取大小
```

2.容器

2.1vector ()

- 定义

```
1 std::vector<T> vec; // 不写任何参数, 创建一个长度为 0, 没有元素的数组
2 std::vector<T> vec(n, val); // 创建一个长度为 n, 全部元素均为 val 的数组, val可以不
    写, 不写时填 T 的默认构造
3 std::vector<T> vec(vec_); // 拷贝构造函数, 把 vec_ 的所有内容拷贝过
4
5 //二维vector的定义, n*m大小
6 std::vector<std::vector<T>> vec(n, std::vector<T>(m));
7 //遍历用for即可
```

- 常用成员

```
1 std::vector<T> vec; // 先提前声明一个 T 类型的 vector
2 vec.push_back(x); // 最后放入一个元素 x
3 vec.pop_back(); // 去除最后一个元素
4 // 中括号随机访问, 这里 i 是一个下标
5 vec[i] = x;
6 vec.back(); // 返回 vector 最后一个元素
7 vec.clear(); //清空vector
8 vec.size(); //获取大小
```

注意: 用push_back会引发动态扩容

2.2set ()

集合 (set), 满足所有元素在里面只会出现至多一次且默认有序

- 定义

```
1 std::set<T> s; // 构造一个空集合
2 std::set<T> s(s_); // 将集合 s_ 的内容复制到 s 中
```

- 常用成员

```
1 std::set<T> s;
2 s.insert(x); // 将 x 放入集合中, 如果已经有了, 不进行任何操作
3 s.count(x); // 返回 x 在集合中的出现次数, 由于集合的特性, 可以理解为判断 x 是否存在集合
    中
4 s.erase(x); // 删除 x
5 s.find(x); // 返回一个指向元素 x 的迭代器, 找不到则返回 s.end()
6 s.empty(); //检查是否为空, 空返回1否则返回0
7 s.size(); //获取大小
```

2.3multiset ()

多重集 (multiset), 也是一个集合, 但是一种元素可以出现多次

- 常用成员

```
1 std::multiset<T> s;
2 s.count(x); // 返回 x 在 s 中的出现次数
3 s.erase(x); // 删除 x 在 s 中的 所有 出现
4 s.extract(x); // 删除 x 在 s 中的一个出现
5 s.find(x); // 返回 x 在 s 中的一个出现的迭代器
6 s.empty(); // 检查是否为空, 空返回1否则返回0
7 s.size(); // 获取大小
```

- set类可以很方便的取出最大值和最小值，在遇到存在多次插入取出，求当前最大最小值时可以考虑

2.4pair()

对组 (pair)，存放两个信息：first, second，相当于一个存放两个变量的类

当一个函数需要返回2个数据的时候，可以选择pair

- 定义

```
1 std::pair<T1, T2> p; // T1和T2b
2 p.first; // 返回对象p1中名为first的公有数据成员
3 p.second; // 返回对象p1中名为second的公有数据成员
```

2.5map ()

map 容器中的元素是按照键的顺序自动排序的，这使得它非常适合需要快速查找和有序数据的场景。

- 定义

```
1 std::map<key_type, value_type> myMap
2 //可声明多维map, 例
3 std::map<int, map<int, int>> myMap;
4 //遍历
5 for (auto [key, value] : f) {
6     std::cout << key << " " << value << "\n";
7 }
```

- 常用成员

```
1 mp[x]; // 返回键x对应的值
2 myMap.erase(key); // 清楚key对应的元素
3 myMap.clear(); // 清空容器
4 myMap.size(); // h
5 myMap.count("Bob"); // key 是否存在存在返回1，否则返回0
```

2.6 stack ()

栈 (stack)，一种后进先出的数据结构，能高效匹配括号，也能处理递归问题等

- 常用成员

```

1 std::stack<T> st;
2 st.push(x) //将元素x压入栈中
3 st.top(); //返回栈顶元素，但不对其进行操作
4 st.pop(); //移除栈顶元素
5 s.empty(); //检查是否为空，空返回1否则返回0
6 s.size(); //获取大小

```

3 模板

3.1 单调队列 滑动窗口

```

1     for(int i=1;i<=n;i++){
2         while((!dq.empty()) && a[dq.back()] <= a[i])
dq.pop_back();
3         dq.push_back(i);
4         if(dq.front() == i - k){
5             dq.pop_front(); //移除过期数据
6         }
7         if(i>=k)
8             cout << a[dq.front()] << " "; //输出窗口每次移动的最大值
9     } //输出长度为k窗口下的最大值

```

子数组最大 / 最小

```

1  ll cur = -1e18;
2  ll ans = -1e18;
3  for(int i = 1;i<=n;i++){
4      cur = max(a[i],cur+a[i]);
5      ans = max(ans,cur);
6  }
7  cout<<ans<<endl; //最大
8
9  ll cur = 0;
10 ll ans = 0;
11 for(int i = 1;i<=n;i++){
12     cur = min(a[i],cur+a[i]);
13     ans = min(ans,cur);
14 }
15 cout<<ans<<endl; //最小

```

3.1.1 找出一定范围内不定长度的最大值

[P1714 切蛋糕 - 洛谷](#)

找出位于m滑块下的不定长最大

简单思考可以得到，若使维护前缀和单调递增，那么得到的答案一定最优

```

1 deque<int> dq;
2 dq.push_back(0); //初始放入索引0
3 int ans = -1e8; //如果出现全负的数据，需要（此题不需要）
4 for (int i = 1; i <= n; i++) {
5     // 维护窗口大小不超过 m，每次最多只有一个过期元素，所以可以用if，但用while更安全
6     // 这里使用if是因为我们知道每次窗口移动一步，最多只有一个过期元素
7     while (!dq.empty() && dq.front() < i - m) {
8         dq.pop_front();
9     }
10    // 更新答案
11    if (!dq.empty()) {
12        ans = max(ans, prefix[i] - prefix[dq.front()]);
13    }
14    // 维护队列单调递增，这里必须用while，因为可能弹出多个
15    while (!dq.empty() && prefix[i] <= prefix[dq.back()]) {
16        dq.pop_back();
17    } //保证前缀和总是单调递增
18    dq.push_back(i);
19 }

```

3.2 单调栈

```

1 for (int i = 1; i <= n; i++)
2 {
3     while ((!st.empty()) && a[st.top()] < a[i]) //单调递减的单调栈
4     {
5         int index = st.top();
6         st.pop();
7         ans[index] = i;
8     }
9     st.push(i);
10 }

```

3.3 二分

```

1 ll r = max;
2 ll l = 1;
3 ll ans = 0;
4
5 while(l <= r){
6     ll temp = 0;
7     ll mid = (l + r) / 2;
8
9     for(int i = 0; i < n; i++){
10         temp += a[i] / mid;
11     }
12
13     if(temp < k){
14         r = mid - 1;
15     }else{
16         l = mid + 1;
17         ans = mid;
18     }
19 }

```

3.4 三分

适用场景：

用于在单峰函数（先增后减）或单谷函数（先减后增）上寻找极值。

- 如果函数是**凸的 (Convex)**（像 U 形）：求最小值。
- 如果函数是**凹的 (Concave)**（像 $\cap$ 形）：求最大值。

注意：函数必须严格单调，若存在平坦区域（平台），三分可能会失效。

3.4.1 实数域三分 (浮点数)

这是最常用的场景（如几何题、物理题）。建议使用**固定迭代次数法**，比 `while(r-l > eps)` 更快且不易死循环。

```

1 // 这里的 check 函数即题目中要求极值的函数
2 double check(double x) {
3     double res = 0;
4     // ... 计算逻辑 ...
5     return res;
6 }
7
8 void solve() {
9     double l = 0, r = 1e9; // 根据题目范围设定
10
11     // 【求最小值】（U形函数）
12     // 循环 100 次可以将精度控制在极高范围，通常优于设置 eps
13     for (int i = 0; i < 100; i++) {
14         double m1 = l + (r - l) / 3;
15         double m2 = r - (r - l) / 3;
16
17         // 如果 m1 处的函数值更小，说明极小值在 m2 左侧（舍弃右边）
18         // 注意：求最大值时，符号改为 >
19         if (check(m1) < check(m2)) {
20             r = m2;
21         } else {
22             l = m1;
23         }
24     }
25
26     // 最终结果 l 和 r 几乎相等，输出 l 或 r 均可
27     printf("%.10f\n", l);
28 }
29

```

3.4.2 整数域三分

当坐标必须是整数时，由于整除截断问题，m1 和 m2 可能会重合。

通用策略：三分将范围缩小到很小（例如区间长度小于 3），然后暴力枚举剩余的几个点。

code C++downloadcontent_copyexpand_less

```
1 long long solve() {
2     long long l = 0, r = 1e9; // 假设求最大值
3
4     // 1. 三分缩小范围，直到区间长度很小（比如 <= 2）
5     while (r - l > 2) {
6         long long m1 = l + (r - l) / 3;
7         long long m2 = r - (r - l) / 3;
8
9         // 求最大值（凹函数 n）
10        // 谁小删谁（m1 比较小，说明峰值在 m1 右侧）
11        if (check(m1) < check(m2)) {
12            l = m1;
13        } else {
14            r = m2;
15        }
16    }
17
18    // 2. 此时区间 [l, r] 只剩下 l, l+1, r 等 2~3 个点
19    // 直接暴力求这几个点的最大值，绝对不会漏，也不会死循环
20    long long ans = -1e18; // 初始极小值
21    for (long long i = l; i <= r; i++) {
22        ans = max(ans, check(i));
23    }
24
25    return ans;
26 }
```

3.5 字符串

3.5.1 统计字符串个数

字串:对于字符串 s 与 t ，如果存在 l 与 r 满足 $1 \leq l \leq r \leq n$ 且 $t = s_l s_{l+1} \cdots s_{r-1} s_r$ ，那么定义 t 为 s 的子串。例如，“garo”是“kangaroo”的子串，而“ko”不是“kangaroo”的子串。

```
1 int cout_find( string s, string p){
2     int cnt = 0;
3     size_t pos = 0;
4     while((pos = s.find(p,pos)) != string::npos){
5         cnt++;
6         pos += 1;
7     }
8     return cnt;
9 }
```

3.5.2统计子序列个数

子序列:对于字符串s与t, 字符串t在字符串s中于子序列的形式出现, 意味着, 字符串t可由s删除若干个字符得到 (也可能是0个)

```
1  int Count(string s,string t){
2      vector<ll> dp(t.length()+1,);
3      dp[0] = 1;
4      for(int i = 0;i<s.length();i++){
5          for(int j = t.length();j;j--){
6              if(s[i] == t[j-1]){
7                  dp[j] += dp[j-1];
8              }
9          }
10     } //dp[j] 的含义是: 目标字符串 t 的“前 j 个字符”在当前扫描过的 s 中作为子序列出现了多少次。
11     return dp[t.length()];
12 }//从后往前遍历t后, dp里分别表示的就是对应位置的前字符串能在s中组成几个
13 //例如 s="babg",t="bag"
14 //遍历完后, dp为[1,2,1,1] dp[1]=2, 表示"b"在s中有两个, dp[2]=1, 表示"ba"在s中有1个
```

3.5.3 字典树

利用字符串的公共前缀来减少查询时间, 最大限度地减少无谓的字符串比较。

```
1  const int N = 100010; // 根据题目总字符长度设定
2  int trie[N][26];      // 假设只存小写字母, 每个节点最多26个子节点
3  int cnt[N];           // 计数器
4  int idx;              // 节点分配器, 从0或1开始
5
6  // 插入字符串
7  void insert(string s) {
8      int p = 0; // p代表当前所在的节点编号, 0是根节点
9      for (int i = 0; i < s.size(); i++) {
10         int u = s[i] - 'a'; // 将字符转化为 0-25 的数字
11         if (!trie[p][u]) trie[p][u] = ++idx; // 如果没有路, 这就新建一条路
12         p = trie[p][u]; // 走到下一个节点
13         cnt[p]++;      // 【重点】这里记录有多少个单词经过了这个节点
14     }
15 }
16
17 // 查询前缀出现次数 (例如查 "ca" 是多少个单词的前缀)
18 int query(string s) {
19     int p = 0;
20     for (int i = 0; i < s.size(); i++) {
21         int u = s[i] - 'a';
22         if (!trie[p][u]) return 0; // 路断了, 说明不存在这个前缀
23         p = trie[p][u];
24     }
25     return cnt[p]; // 返回经过这个节点的数量
26 }
```


3.5.4 LCS && LIS

最长公共子序列(LCS)

```
1  int dp[5010][3010];
2  string s, t;
3  cin >> s >> t;
4
5  for (int i = 1; i <= s.length(); i++)
6  {
7      for (int j = 1; j <= t.length(); j++)
8      {
9          if (s[i-1] == t[j-1])
10         {
11             dp[i][j] = dp[i - 1][j - 1] + 1;
12         }
13         else
14         {
15             dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
16         }
17     }
18 }
19 cout<<dp[s.length()][t.length()]; //输出最长公共子序列长度
20
21 //反向追踪出最长的公共子序列
22 string res = "";
23 int i = s.length(), j = t.length();
24 while(i>0 && j>0){
25     if(s[i - 1] == t[j-1] ){
26         res+=s[i-1];
27         i--,j--;
28     }else{
29         if(dp[i-1][j] >= dp[i][j-1]) i--;
30         else j--;
31     }
32 }
33 reverse(res.begin(),res.end());
34 cout<<res;
35
```

最长上升子序列(LIS)

设计 dp_i 为以 a_i 为结尾的最长上升子序列，计算时，尝试将 a_i 接到之前的最长不下降子序列后面

```

1  dp[1] = 1;
2  ll ans = 1;
3  for(int i = 2; i <= n; i++){
4      dp[i] = 1;
5      for(int j = 1; j < i; j++){
6          if(a[j] < a[i]){
7              dp[i] = max(dp[i], dp[j] + 1);
8              ans = max(ans, 1LL * dp[i]);
9          }
10     }
11 }
12 cout << ans;

```

n^2 的算法如果对 $1e5$ 及以上的数据来说有点慢，我们可以进行优化，可优化为 $O(n \log n)$ ，如下

```

1  vector<int> low(n + 1, 0);
2  int len = 0;
3  for (int i = 1; i <= n; i++)
4  {
5      if (b[i] > low[len])
6      {
7          len++;
8          low[len] = b[i];
9      }
10     else
11     {
12         int idx = lower_bound(low.begin() + 1, low.begin() + len + 1, b[i]) -
low.begin();
13         low[idx] = b[i];
14     }
15 }
16
17 cout << len;

```

解释: low_i 表示长度为 i 的最长上升子序列；我们从1开始遍历到 n ，如果遇到当前数组的数值大于 low 中最后的元素，我们就可以把当前的数值接到后面；如果遇到严格小于 low 中最后的元素，我们就可以将其替换到第一个大于他的位置上，可以证明，这是更优的，因为如果后面的值越小，就更容易接上更多的值

求 **LCS** 的问题部分也可转化为求 **LIS** 的问题，例如，如果对应两个数组中的元素范围相同且每个数只出现一次，我们就可以把其中一个数组当作基准来调整另一个数组中的元素；

更具体的说，如果现在给你两个数组 a , b ，他们都是 n 的排列，让你求出 a 和 b 的最长公共子序列，我们就可以以 a 为基准，对 b 进行映射，那么问题就**等价**于对映射后的 b 求最长上升子序列

如下

```

1  int n;
2  cin >> n;
3  vector<int> pos(n+1, 0);
4  vector<int> b(n + 1, 0);
5  for (int i = 1; i <= n; i++)
6  {
7      int a;

```

```

8         cin>>a;
9         pos[a] = i;
10    }
11    for (int i = 1; i <= n; i++)
12    {
13        cin >> b[i];
14        b[i] = pos[b[i]];
15    }
16
17    vector<int> low(n + 1, 0);
18    int len = 0;
19    for (int i = 1; i <= n; i++)
20    {
21        if (b[i] > low[len])
22        {
23            len++;
24            low[len] = b[i];
25        }
26        else
27        {
28            int idx = lower_bound(low.begin()+1, low.begin()+len+1, b[i]) -
low.begin();
29            low[idx] = b[i];
30        }
31    }
32
33    cout << len;
34

```

3.5.5 KMP

KMP 的核心任务是解决单模式串在主串中的匹配问题。当你用模式串 P 去匹配主串 T 时，如果中途在 $P[j]$ 处失配了，暴力算法会把主串指针回溯。而 KMP 告诉你：主串指针永远不回头，只移动模式串指针

核心数组：next（或 pi 数组）定义：next[i] 表示前缀 $P[0 \dots i]$ 的最长相等**前后**缀的长度（不能是它本身）。用途：当模式串在 $P[j]$ 失配时，说明前面的 $P[0 \dots j - 1]$ 已经匹配成功。由于有 next 数组的存在，我们可以直接把模式串向右滑，让 $P[\text{next}[j - 1]]$ 对齐刚才失配的位置继续比对。

模板：

```

1  // 计算最长相等前后缀 pi 数组（即通常说的 next 数组）
2  vector<int> compute_pi(const string& s) {
3      int n = s.size();
4      vector<int> pi(n, 0);
5      for (int i = 1; i < n; i++) {
6          int j = pi[i - 1];
7          while (j > 0 && s[i] != s[j]) j = pi[j - 1];
8          if (s[i] == s[j]) j++;
9          pi[i] = j;
10     }
11     return pi;
12 }
13
14 // KMP 匹配：返回所有匹配成功的起始下标（0-indexed）
15 vector<int> kmp_match(const string& text, const string& pattern) {
16     if (pattern.empty()) return {};

```

```

17     vector<int> pi = compute_pi(pattern);
18     vector<int> matches;
19     int j = 0;
20     for (int i = 0; i < text.size(); i++) {
21         while (j > 0 && text[i] != pattern[j]) j = pi[j - 1];
22         if (text[i] == pattern[j]) j++;
23         if (j == pattern.size()) {
24             matches.push_back(i - j + 1);
25             j = pi[j - 1]; // 继续找下一个匹配
26         }
27     }
28     return matches;
29 }

```

KMP扩展

1. **寻找最小循环节** (极其经典) 对于一个长度为 n 的字符串 S , 计算出它的 pi 数组。定理: 若 $n \pmod{n - pi[n - 1]} == 0$ 且 $pi[n - 1] > 0$, 则 S 具有常数周期的循环节, 其最小循环元长度为 $L = n - pi[n - 1]$, 循环次数为 $\frac{n}{L}$ 。如果不能整除, 说明它尾部缺了一点。但 $n - pi[n - 1]$ 依然代表补齐后可能形成的最小周期。

```

1  int L = n - pi[n - 1];
2
3  // 3. 判定是否能完美整除
4  if (n % L == 0 && pi[n - 1] > 0) {
5      cout << n / L << "\n";
6  } else {
7      cout << 1 << "\n";
8  }

```

2. **前缀出现次数统计** 怎么在 $O(n)$ 时间内求出字符串 S 的所有前缀在 S 自身中出现了多少次? 每一个 $pi[i]$ 都代表一个前缀的结束。我们可以建立一张图, 从 i 向 $pi[i - 1]$ 连边。这本质上是一棵 KMP 树 (失配树)。通过在树上从叶子到根进行拓扑求和 (或者简单的倒序递推 $ans[pi[i - 1]] += ans[i]$), 就可以一次性统计出所有前缀的全局出现次数。

```

1  // 统计 S 的所有前缀在 S 自身中的出现次数
2  vector<int> count_prefix_occurrences(const string& s) {
3      int n = s.size();
4      vector<int> pi = compute_pi(s); // 使用之前的 KMP 模板
5
6      // ans[len] 表示长度为 len 的前缀出现的总次数
7      vector<int> ans(n + 1, 0);
8
9      // 步骤 1: 每个位置先自主贡献 1 次
10     for (int i = 0; i < n; i++) {
11         ans[i + 1] = 1;
12     }
13
14     // 步骤 2: 从大到小倒序递推, 相当于在失配树上从叶子向根节点求后缀和
15     for (int i = n; i > 0; i--) {
16         if (pi[i - 1] > 0) {
17             ans[pi[i - 1]] += ans[i];
18         }
19     }
20
21     // 此时 ans[len] 里存的就是长度为 len 的前缀在全局出现的绝对次数

```

```

22     return ans;
23 }

```

3.5.6 exKMP(Z函数)

exKMP 是 KMP 的全方位升级版。它的核心任务是：求文本串 T 的每一个后缀与模式串 P 的最长公共前缀 (LCP)。

核心数组： z 数组与 ext 数组竞赛中为了代码复用，通常分为两步：

1. z 数组 (对 P 自身求)： $z[i]$ 表示以 $P[i]$ 开头的后缀 $P[i \dots m-1]$ 与整个 P 的 LCP 长度。
2. ext 数组 (T 与 P 匹配)： $ext[i]$ 表示以 $T[i]$ 开头的后缀 $T[i \dots n-1]$ 与整个 P 的 LCP 长度。

工作机制： Z-box (匹配盒子) exKMP 在维护过程中，会记录一个当前拓展到最右端的匹配区间 $[l, r]$ (即 $T[l \dots r]$ 完全匹配了 $P[0 \dots r-l]$)。

- 当我们计算 i 位置时，如果 $i \leq r$ ，根据对称性， $T[i \dots r]$ 的信息完全等于 $P[i-l \dots r-l]$ 。
- 我们可以直接利用之前算好的 $z[i-l]$ 来一刀切掉大段的重复匹配，直接从不需要比对的地方继续向外暴力外扩。

模板：

```

1  // 计算 P 自身的 Z 数组 (即 exKMP 中的 nxt 数组)
2  vector<int> compute_z(const string& p) {
3      int m = p.size();
4      vector<int> z(m, 0);
5      z[0] = m; // 自全匹配
6      for (int i = 1, l = 0, r = 0; i < m; ++i) {
7          if (i <= r) z[i] = min(r - i + 1, z[i - l]);
8          while (i + z[i] < m && p[z[i]] == p[i + z[i]]) ++z[i];
9          if (i + z[i] - 1 > r) {
10             l = i;
11             r = i + z[i] - 1;
12         }
13     }
14     return z;
15 }
16
17 // 计算 T 的每个后缀与 P 的 LCP (ext 数组)
18 vector<int> exkmp(const string& t, const string& p) {
19     int n = t.size(), m = p.size();
20     vector<int> z = compute_z(p);
21     vector<int> ext(n, 0);
22     for (int i = 0, l = 0, r = 0; i < n; ++i) {
23         if (i <= r) ext[i] = min(r - i + 1, z[i - l]);
24         while (i + ext[i] < n && ext[i] < m && p[ext[i]] == t[i + ext[i]])
25             ++ext[i];
26         if (i + ext[i] - 1 > r) {
27             l = i;
28             r = i + ext[i] - 1;
29         }
30     }
31     return ext;

```

exKMP扩展：

1. 任意位置的周期判定

KMP 只能很方便地判定整个串的周期。而 exKMP 拥有无死角的视角：

- 如果你想判断以 i 开头的后缀是否是原串的一个周期，只需看 $z[i]$ 是否等于 $n - i$ 。如果是，说明 $S[i \dots n - 1]$ 与原串前缀完全一致，意味着整个串在 i 处发生了完美的错位重合。

2. 字符串拼接与前后缀重合问题

比如题目要求：找出所有既是 S 的前缀、又是 S 的后缀、并且在 S 中间还出现过至少一次的子串。 z 数组天然包含了“既是前缀又是后缀”的信息（只要 $i + z[i] == n$ 即可）。我们只需要遍历中间部分的 z 值，看看有没有大于等于当前后缀长度的值即可。配合线段树或简单的树状数组，能秒杀各种复杂的子串计数问题。

、-----

解答：

如何判断“是后缀”？

如果从位置 i 开始的匹配一直顶到了字符串的最后一个字符，说明它就是原串的一个后缀。

Z 数组翻译： $i + z[i] == n$ 。此时这个后缀的长度就是 $z[i]$ 。

如何判断“在中间出现过”？ 如果这个长度为 $z[i]$ 的后缀/前缀，在 i 之前还出现过，说明它在中间也有过匹配。这意味着在 $1 \dots i - 1$ 之间，必然存在某个位置 j ，使得从 j 开始的匹配长度至少能覆盖 $z[i]$ 。

Z 数组翻译： 存在 $j < i$ ，使得 $z[j] \geq z[i]$ 。

code:

```
1 // 引入计算 z 数组的模板（与上文一致）
2 vector<int> compute_z(const string& s) {
3     int n = s.size();
4     vector<int> z(n, 0);
5     z[0] = n;
6     for (int i = 1, l = 0, r = 0; i < n; ++i) {
7         if (i <= r) z[i] = min(r - i + 1, z[i - l]);
8         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i];
9         if (i + z[i] - 1 > r) {
10             l = i;
11             r = i + z[i] - 1;
12         }
13     }
14     return z;
15 }
16
17 string find_password(const string& s) {
18     int n = s.size();
19     vector<int> z = compute_z(s);
20
21     int max_z = 0; // 记录在当前 i 之前，出现过的最大 z 值
22     int ans_len = 0; // 记录满足条件的最长子串长度
23
24     for (int i = 1; i < n; i++) {
25         // 条件 1: 是后缀 (i + z[i] == n)
26         // 条件 2: 在中间出现过 (max_z >= z[i])
27         if (i + z[i] == n && max_z >= z[i]) {
28             ans_len = max(ans_len, z[i]);
29         }
30
31         // 更新之前出现过的最大匹配长度
32         max_z = max(max_z, z[i]);
33     }
34 }
```

```

35     if (ans_len > 0) {
36         return s.substr(0, ans_len);
37     } else {
38         return "Just a legend"; // 题目要求的无解输出
39     }
40 }

```

任务：给定长度为 n 的字符串 S ，我们在第 i 个位置切一刀，把前缀 $S[0 \dots i - 1]$ 拼接到最后 $S[i \dots n - 1]$ 的后面。怎么快速判断拼接后的新串是否和原串 S 依然一模一样？

核心逻辑：假设我们在下标 i 处切一刀，原串 S 被分成了两部分：

- 前部 A: $S[0 \dots i - 1]$ ，长度为 i 。
- 后部 B: $S[i \dots n - 1]$ ，长度为 $n - i$ 。拼接后的新串是 $B + A$ 。我们要求 $B + A == S$ 。
既然 S 本身就是 $A + B$ 构成的，要想 $B + A == A + B$ ，必须同时满足两个极其严苛的条件：
 1. B 必须和 S 的前部完全一样：也就是说，后缀 $S[i \dots n - 1]$ 必须和原串前缀匹配。
 - Z 数组翻译: $z[i] == n - i$
 2. A 必须和 S 的后部完全一样：也就是说，原串前缀 $S[0 \dots i - 1]$ 必须和后缀 $S[n - i \dots n - 1]$ 匹配。
 - Z 数组翻译: $z[n - i] \geq i$ (因为 $S[n - i]$ 开头的后缀的最长公共前缀至少得覆盖长度 i)。

```

1 // 纯 Z 数组原地判定法
2 vector<int> find_valid_cuts_inplace(const string& s) {
3     int n = s.size();
4     vector<int> z = compute_z(s); // 只需要对原串求一次 Z 数组
5     vector<int> valid_cuts;
6
7     // 0 处切分（不切）默认算一种，如果要排除可以从 1 开始
8     if (n > 0) valid_cuts.push_back(0);
9
10    for (int i = 1; i < n; i++) {
11        // 条件1: 后部 B (长度 n-i) 必须和 S 的前缀匹配
12        bool cond1 = (z[i] == n - i);
13        // 条件2: 前部 A (长度 i) 必须和 S 的后缀(起点为 n-i)匹配
14        bool cond2 = (z[n - i] >= i);
15
16        if (cond1 && cond2) {
17            valid_cuts.push_back(i);
18        }
19    }
20    return valid_cuts;
21 }

```

3.5.7 马拉车算法（处理最长回文串）

马拉车算法（Manacher's Algorithm）是专门为解决“最长回文子串”问题而生的。它通过巧妙地利用回文串的**对称性**，跳过了大量重复的字符比较，将时间复杂度从原本的 $O(n^2)$ 压缩到了极其极致的 $O(n)$ 。

```

1 void solve()
2 {
3     string s;

```

```

4      cin >> s;
5
6      // 1. 预处理字符串
7      // 在每个字符之间插入 '#', 并在首尾加入不同的特殊字符 (如 '^' 和 '$')
8      // 这样既统一了奇偶长度问题, 又避免了中心扩展时的数组越界检查
9      string t = "^#";
10     for(char c : s) {
11         t += c;
12         t += '#';
13     }
14     t += '$';
15
16     int n = t.length();
17     vector<int> p(n, 0); // p[i] 记录以 t[i] 为中心的最长回文半径
18     int C = 0; // 当前能够向右扩展最远的回文串的中心
19     int R = 0; // 当前能够向右扩展最远的回文串的右边界
20     int max_len = 0;
21
22     for(int i = 1; i < n - 1; i++) {
23         // 2. 利用已知回文串的对称性, 初始化 p[i]
24         // 如果 i 在 R 的左侧, 可以利用对称点 (2 * C - i) 的信息加速
25         if (i < R) {
26             p[i] = min(R - i, p[2 * C - i]);
27         }
28
29         // 3. 尝试继续向两边扩展 (由于首尾有 ^ 和 $ 挡着, 无需判断越界)
30         while (t[i + 1 + p[i]] == t[i - 1 - p[i]]) {
31             p[i]++;
32         }
33
34         // 4. 如果新找到的回文串右边界超过了当前的 R, 则更新 C 和 R
35         if (i + p[i] > R) {
36             C = i;
37             R = i + p[i];
38         }
39
40         // p[i] 恰好等于原字符串中对应回文串的总长度
41         max_len = max(max_len, p[i]);
42     }
43
44     cout << max_len << "\n";
45 }

```

进制转换

十进制转2-16进制

```

1  int n;
2  string s;
3  int x;
4  char c;
5  for(int i = 2; i <= 16; i++){
6      int nu = n;

```



```

7     while(nu){
8         x = nu%i;
9         if(x<10){
10            c = x+'0';
11        }else{
12            c = x+'A'-10;
13        }
14        nu/=i;
15        s = s +c;
16    }
17    cout<<s<<endl;
18 }

```

3.6 搜索 && 数据结构

```

1  //lambda表达式:
2  auto dfs = [&](auto &&self,int u,int fa) -> void{
3      ....
4      //调用时
5      self(self,v,u);
6      return;
7  };
8  //第一次调用
9  dfs(dfs,1)
10
11
12 void dfs(int u, int fa) {
13     for (int v : adj[u]) {
14         if (v == fa)
15             continue;
16         dfs(v, u);
17     }
18 }
19
20 //二者等价
21
22 void dfs(int u,int fa){
23     for(int v = 0;v<adj[u].size();u++){
24         if(adj[u][v] == fa)
25             continue;
26         dfs(adj[u][v],u);
27     }
28 }
29
30
31 //统计子树+公共祖先倍增
32
33 vector<ll> in(N),out(N); //统计进出x
34 ll tim =0;
35 vector<ll> sz(N,0); //统计子树大小
36 vector<vector<ll>> up(N,vector<ll>(LOG,0)); //倍增祖先 up[u][i]表示u向上跳2^i
    步到达的节点
37 vector<ll> depth(N,0); //统计深度
38 void dfs(int u,int fa,int d){
39     in[u] = ++tim;

```

```

40     depth[u] = d;
41     sz[u] = 1;
42
43     up[u][0] = fa;
44     for(int i = 1; i < LOG; i++){
45         if(up[u][i-1] != 0){
46             up[u][i] = up[up[u][i-1]][i-1];
47         }else{
48             up[u][i] = 0;
49         }
50     }
51
52     for(int v:adj[u]){
53         if(v == fa) continue;
54         dfs(v, u, d+1);
55         sz[u] += sz[v]; //累加子树大小
56     }
57     out[u] = ++tim;
58 }
59
60 bool isAncestor(int u, int v){ //判断v是否是u的祖先
61     return in[u] <= in[v] && out[v] <= out[u];
62 }
63
64 // 找到 v 的祖先中，是 u 的直接子节点的那个点
65 // 前提: u 是 v 的严格祖先
66 int getChildTowards(int u, int v) { //获取u-1
67
68     for (int i = LOG - 1; i >= 0; i--) {
69         // 如果跳一步之后，深度仍然比 u 大（说明还在 u 下面），就往上跳
70         if (up[v][i] != 0 && depth[up[v][i]] > depth[u]) {
71             v = up[v][i];
72         }
73     }
74     return v;
75 }
76
77 //寻找u,v的lca
78 // 假设 MAX_LOG 一般取 20 (因为  $2^{20} > 10^5$ )
79 int get_lca(int u, int v) {
80     // 步骤 1: 始终保持 u 是更深的那个节点（方便后面把 u 往上提）
81     if (depth[u] < depth[v]) {
82         swap(u, v);
83     }
84
85     // 步骤 2: 让 u 往上跳，直到 u 和 v 处于同一深度
86     // 从大步到小步尝试
87     for (int i = MAX_LOG; i >= 0; i--) {
88         // 如果 u 跳  $2^i$  步之后，高度仍然 >= v 的高度，那就跳
89         // (注: 如果跳过头了，到了 0 号节点，depth[0] 一般是 0，条件自然不成立)
90         if (depth[up[u][i]] >= depth[v]) {
91             u = up[u][i];
92         }
93     }
94
95     // 步骤 3: 特判。如果齐平后 u 和 v 重合了，说明原来的 v 就是 u 的祖先
96     if (u == v) {
97         return u;

```

```

98     }
99
100    // 步骤 4: u 和 v 同时往上跳, 寻找 LCA 的“正下方”那个节点
101    for (int i = MAX_LOG; i >= 0; i--) {
102        // 如果 u 和 v 跳  $2^i$  步后, 到达的节点【不一样】
103        // 说明还没有到达 LCA (或者还没越过 LCA), 那就可以跳!
104        if (up[u][i] != up[v][i]) {
105            u = up[u][i];
106            v = up[v][i];
107        }
108    }
109
110    // 步骤 5: 循环结束后, u 和 v 必定处于 LCA 的直接子节点位置
111    // 所以它们的父节点 up[u][0] 就是 LCA
112    return up[u][0];
113 }

```

```

1 void bfs(int st) {
2     vector<int> vis(N, 0);
3     queue<int> q;
4     q.push(st);
5     vis[st] = 1; //标记已经访问过的
6     while (!q.empty()) {
7         int u = q.front();
8         q.pop(); //访问后移除
9         for (int v : adj[u]) {
10             if (vis[v])
11                 continue;
12             q.push(v); //将以u为节点的子节点存入队列, 继续遍历
13             vis[v] = 1; //标记
14         }
15     }
16 }

```

判环

1. 无向图: 如果遇到一个已经访问过的点, 那么说明有环

- dfs: 递归访问, 记录vis。如果邻居节点v访问过且不是父节点, 则有环
- 并查集: 遍历每一条边, 如果 `find(u) == find(v)`, 说明有环
- bfs: 同理dfs

2. 有向图: 不能简单看是否访问过

- dfs+三色标记:
 - 0表示未访问, 1表示正在递归中, 没有判断完, 2表示已完全递归完毕, 该节点不在环内

```

1  bool dfs(ll u){
2      vis[u] = 1;
3      for(ll &v : adj[u]){
4          if(vis[v]==1){
5              return true;
6          }
7          if(vis[v]==0 && dfs(v)) return true;
8      }
9
10     vis[u] = 2;
11     return false;
12 }

```

- 拓扑

- 逻辑:

1. 统计所有点入度。
2. 将入度为 0 的点入队。
3. 不断弹出队首，将其邻居入度减 1。若邻居入度减为 0，则入队。

- 判定：如果最后处理过的节点总数 < 图中节点总数，说明剩下的点入度永远减不到 0（互相死锁），有环。

- 优点：逻辑清晰，还能顺便求出拓扑序。

```

1  vector<ll> deg(n+1,0)//存节点入度
2  for(int i = 1;i<=n;i++){
3      for(ll v : adj[i]){
4          deg[v]++;
5      }
6  }
7
8  queue<ll> q;
9  for(int i = 1;i<=n;i++){
10     if(deg[i]==0) q.push(i);
11 }
12
13 ll count = 0;
14
15 while(!q.empty()){
16     ll u = q.front();
17     q.pop();
18     count++;
19
20     for(ll v : adj[u]){
21         deg[v]--;
22         if(deg[v]==0) q.push(v);
23     }
24 }
25
26 if(count == n){
27     //无环
28 }else{
29     //有环
30 }

```

3.负权环

- 详情见最短路中的SPFA

最短路

dijkstra(处理非负权边的最短路)

标准模板

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  // 使用 long long 防止路径总和爆 int
5  using ll = long long;
6
7  // 定义无穷大, 注意不要用 INT_MAX, 防止相加溢出
8  const ll INF = 0x3f3f3f3f3f3f3f;
9  // 或者直接 const ll INF = 1e18;
10
11 const int N = 100005; // 根据题目最大节点数修改
12 const int M = 200005; // 根据题目最大边数修改
13
14 // 邻接表存图: vector<pair<目标点, 权值>>
15 struct Edge {
16     int to;
17     ll w;
18 };
19 vector<Edge> adj[N];
20
21 // dist[i] 存储起点到 i 的最短距离
22 ll dist[N];
23
24 // 标记数组 (可选, 但在堆优化中用于剪枝)
25 // bool vis[N];
26
27 // n: 节点数, s: 起点
28 void dijkstra(int n, int s) {
29     // 1. 初始化距离为无穷大
30     for (int i = 1; i <= n; i++) {
31         dist[i] = INF;
32         // vis[i] = false;
33     }
34
35     // 2. 起点距离设为 0
36     dist[s] = 0;
37
38     // 3. 优先队列 (小根堆): 存储 {当前距离, 节点编号}
39     // greater 让 pair 按照 first 从小到大排序
40     priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll,
41 int>>> pq;
42
43     // 把起点放入队列
44     pq.push({0, s});
45
46     while (!pq.empty()) {
```

```

47     auto [d, u] = pq.top();
48     pq.pop();
49
50     // 【关键剪枝】：懒惰删除
51     // 如果当前取出的距离 d 大于已经更新过的最短距离 dist[u],
52     // 说明这个节点是旧的、无效的信息，直接跳过。
53     if (d > dist[u]) continue;
54
55     // 如果需要 vis 数组：
56     // if (vis[u]) continue;
57     // vis[u] = true;
58
59     // 遍历 u 的所有邻居
60     for (auto& edge : adj[u]) {
61         int v = edge.to;
62         ll w = edge.w;
63
64         // 【松弛操作】：如果经由 u 到 v 更近
65         if (dist[u] + w < dist[v]) {
66             dist[v] = dist[u] + w;
67             pq.push({dist[v], v}); // 将更新后的 v 放入队列
68         }
69     }
70 }
71 }
72
73 int main() {
74     // 加速 I/O
75     ios::sync_with_stdio(false);
76     cin.tie(0);
77
78     int n, m, s;
79     // 输入：节点数，边数，起点
80     cin >> n >> m >> s;
81
82     // 建图
83     for (int i = 0; i < m; i++) {
84         int u, v;
85         ll w;
86         cin >> u >> v >> w;
87         // 有向图
88         adj[u].push_back({v, w});
89
90         // 如果是无向图，加上下面这句：
91         // adj[v].push_back({u, w});
92     }
93
94     // 运行算法
95     dijkstra(n, s);
96
97     // 输出结果
98     for (int i = 1; i <= n; i++) {
99         if (dist[i] == INF) {
100             cout << -1 << " "; // 无法到达
101         } else {
102             cout << dist[i] << " ";
103         }
104     }

```

```

105     cout << endl;
106
107     return 0;
108 }

```

全源最短路(处理含负权边) (SPFA)

- 核心思想：首先创建一个虚拟超级 0 节点，向所有节点连一条权值为 0 的边，跑一次 *SPFA* 求出势能函数后，利用他把所有边转换成非负权边，接着按照题意把要求的最短路用 *dijkstra* 求出即可

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  using ll = long long;
4  #define endl '\n'
5
6  const ll INF = 1e18; // 设置一个足够大的无穷大值，防止累加时溢出
7
8  // 边的结构体
9  struct node {
10     ll to; // 目标节点
11     ll w; // 边权
12 };
13
14 int main() {
15     // -----
16     // 0. 基础设置与建图
17     // -----
18     ios::sync_with_stdio(0); // 优化 C++ 输入输出流，防止大数据读写超时
19     cin.tie(0);
20
21     ll n, m;
22     cin >> n >> m;
23
24     // 使用 vector 的 vector，兼容所有 C++ 标准，避免 VLA（变长数组）报错
25     vector<vector<node>> adj(n + 1);
26
27     for (int i = 1; i <= m; i++) {
28         ll u, v, w;
29         cin >> u >> v >> w;
30         adj[u].push_back({v, w}); // 构建有向图
31     }
32
33     // -----
34     // 1. SPFA 求势能数组 h（隐式建立虚拟节点 0）
35     // -----
36     vector<ll> cnt(n + 1, 0); // cnt[i] 记录节点 i 入队的次数，用于判负环
37     vector<bool> vis(n + 1, false); // vis[i] 表示节点 i 当前是否在队列中（SPFA
    核心）
38
39     // h 初始必须为 0！等价于虚拟节点 0 到各点的距离为 0
40     vector<ll> h(n + 1, 0);
41     queue<ll> q;
42
43     // 隐式从虚拟源点出发：将所有真实节点压入队列，初始距离(势能)全为 0

```

```

44     for (int i = 1; i <= n; i++) {
45         q.push(i);
46         vis[i] = true; // 标记已在队列中
47         cnt[i] = 1;    // 相当于所有点已经入队 1 次
48     }
49
50     // SPFA 主循环
51     while (!q.empty()) {
52         auto u = q.front();
53         q.pop();
54         vis[u] = false; // 节点出队，取消标记
55
56         for (auto &edge : adj[u]) {
57             int v = edge.to;
58             int w = edge.w;
59
60             // 松弛操作
61             if (h[v] > h[u] + w) {
62                 h[v] = h[u] + w;
63
64                 // 只有当 v 不在队列中时，才需要将其入队
65                 if (!vis[v]) {
66                     vis[v] = true;
67                     cnt[v]++; // 记录入队次数
68
69                     // 如果某个点入队次数超过图的总节点数 n，说明在无限绕负权环
70                     if (cnt[v] > n) {
71                         cout << -1 << endl;
72                         return 0; // 发现负环，直接结束程序
73                     }
74
75                     // q.push 必须放在 if(!vis[v]) 内部！防止重复无意义入队
76                     q.push(v);
77                 }
78             }
79         }
80     }
81
82     // 将所有边转换为非负
83     for (int i = 1; i <= n; i++) {
84         for (auto &edge : adj[i]) {
85             edge.w = edge.w + h[i] - h[edge.to];
86         }
87     }
88
89     // 后面根据题目进行dijkstra即可，注意最后要对边权进行还原
90     // 真实 w = dist[j] + h[j] - h[i]
91
92     return 0;
93 }

```


分册图+回溯寻找路径

[P1266 [BalticOI 2002](#)] 速度限制 - 洛谷

1. 分册图

- **分层图最短路**，本质上是**图论与动态规划（DP）的结合**（所以也常被称为“状态机最短路”）。
- 因为到达某一个节点时会因为上一个节点的结果不同而可能导致当前节点的结果不同，那么这种情况下我们需要对 *dist* 数组进行升维，二维或者三维，根据题目限制条件而定
 - 一般题目带有“**状态/油量/速度/免费次数/打折券**”等带有限制性条件时，一般都需要分层图

2. 回溯找最短路对应的节点

- 可根据 *dist* 创建对应数量和维度的 *pre* 数组，用来存放谁更新了当前最优的状态，最后倒序查找即可

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  using ll = long long;
4  #define endl '\n'
5
6  const ll INF = 1e18;
7
8  struct Node{
9      int to;
10     int l;
11     int v;
12 };
13
14 int main(){
15     ios::sync_with_stdio(0);
16     cin.tie(0);
17     int n,m,d;
18     cin>>n>>m>>d;
19     vector<Node> adj[n];
20     for(int i = 1;i<=m;i++){
21         int u,v,s,l;
22         cin>>u>>v>>s>>l;
23         adj[u].push_back({v,l,s});
24     }
25
26     //本题的边权，也就是时间并非固定不变，所有需要再开一维来进行表示，dist[u][se]表示到达u节点且速度为se时的最短路径
27     vector<vector<double>> dist(n,vector<double>(505,INF));
28
29     priority_queue<tuple<double,ll,ll>,vector<tuple<double,ll,ll>>,greater<tuple<double,ll,ll>>> pq;
30     vector<vector<int>> pre_pos(n,vector<int>(505,-1)); //表示到达对应状态的上一个路口
31     vector<vector<int>> pre_sp(n,vector<int>(505,-1)); //表示到达对应状态的上一个速度
32     pq.push({0.0,70,0});
33     dist[0][70] = 0;
34
35     while(!pq.empty()){
36         auto [t,se,u] = pq.top();
```

```

36     pq.pop();
37
38     if(t>dist[u][se]) continue;
39
40     for(auto &edge : adj[u]){
41         int v = edge.to;
42         int l = edge.l;
43         int se1 = edge.v;
44         double t1 = 0.0;
45
46         if(se1 == 0){
47             t1 = (double)l / se;
48             se1 = se;
49         }else{
50             t1 = (double)l / se1;
51         }
52
53         if(dist[v][se1]>dist[u][se]+t1){
54             dist[v][se1] = dist[u][se] + t1;
55             pq.push({dist[v][se1],se1,v});
56             pre_pos[v][se1] = u;
57             pre_sp[v][se1] = se;
58         }
59     }
60 }
61
62 double minn = INF;
63 vector<int> ans;
64 int sp = -1;
65 for(int i = 0;i<=500;i++){
66     if(minn>dist[d][i]){
67         minn = dist[d][i];
68         sp = i;
69     }
70 }
71
72 int cur_u = d;
73 int cur_sp = sp;
74
75 while(cur_u != -1){
76     ans.push_back(cur_u);
77
78     int next_u = pre_pos[cur_u][cur_sp];
79     int next_sp = pre_sp[cur_u][cur_sp];
80
81     cur_u = next_u;
82     cur_sp = next_sp;
83 }
84 reverse(ans.begin(),ans.end());
85
86 for(int i = 0;i<ans.size();i++){
87     cout<<ans[i]<<" ";
88 }
89
90 return 0;
91 }

```

Floyd

该算法可求任意两点的之间的最短长度，但复杂度过高，一般只在n较小的情况下使用或优化

```
1 for (k = 1; k <= n; k++) {
2     for (x = 1; x <= n; x++) {
3         for (y = 1; y <= n; y++) {
4             f[x][y] = min(f[x][y], f[x][k] + f[k][y]);
5         }
6     }
7 } //f[x][y]表示x到y的最短路程
```

处理任意k个点中任意两点间的最短路

```
1 void solve()
2 {
3     int n, m;
4     cin >> n >> m;
5
6     // 使用 pair 存储图: {目标节点 v, 边权 w}
7     vector<vector<pair<int, ll>>> adj(n + 1);
8     for(int i = 0; i < m; i++){
9         int u, v;
10        ll w;
11        cin >> u >> v >> w;
12        adj[u].push_back({v, w});
13        adj[v].push_back({u, w}); // 无向图, 双向建边
14    }
15
16    int k;
17    cin >> k;
18
19    // dist 记录某个节点距离离它最近的特殊点的距离
20    vector<ll> dist(n + 1, INF);
21    // color 记录离该节点最近的特殊点是哪一个 (即势力的 ID)
22    vector<int> color(n + 1, 0);
23
24    // priority_queue 存储: {距离, 当前节点ID}, 默认大根堆, 用 greater 变成小根堆
25    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll,
26    int>>> pq;
27
28    // 将所有特殊点作为多源 BFS/Dijkstra 的起点同时入队
29    for(int i = 0; i < k; i++){
30        int start_node;
31        cin >> start_node;
32        dist[start_node] = 0;
33        color[start_node] = start_node; // 自己染自己的色
34        pq.push({0, start_node});
35    }
36
37    ll ans = INF;
38
39    // 开始多源 Dijkstra 扩张
```

```

39     while(!pq.empty()){
40         ll d = pq.top().first;
41         ll u = pq.top().second;
42         pq.pop();
43
44         // 剪枝: 如果当前取出的距离不是最优的, 直接跳过
45         if(d > dist[u]) continue;
46
47         for(auto &edge : adj[u]){
48             ll v = edge.first;
49             ll w = edge.second;
50
51             // 状态 1: 目标节点 v 还是无主之地 (没被染色)
52             if(color[v] == 0){
53                 color[v] = color[u]; // 插上和 u 一样的旗帜
54                 dist[v] = dist[u] + w;
55                 pq.push({dist[v], v});
56             }
57             // 状态 2: 目标节点 v 已经被别的势力占领了! 两军相遇!
58             else if(color[v] != color[u]){
59                 // 不入队, 直接用相遇距离更新全局最小答案
60                 ans = min(ans, dist[u] + dist[v] + w);
61             }
62             // 状态 3: 目标节点 v 已经是自己势力的地盘了, 看看能不能找到更短的巡逻路线
63             else if(color[v] == color[u]){
64                 if(dist[v] > dist[u] + w){
65                     dist[v] = dist[u] + w;
66                     pq.push({dist[v], v});
67                 }
68             }
69         }
70     }
71
72     cout << ans << "\n";
73 }

```

并查集

专门用来处理一些**不相交集合的合并与查询**问题

```

1  /**
2   * 并查集 (DSU) 模板
3   * 包含: 路径压缩 + 按大小合并
4   * 复杂度:  $O(\alpha(n)) \approx O(1)$ 
5   */
6  struct DSU {
7      std::vector<int> parent;
8      std::vector<int> siz; // 记录每个集合的大小
9      ll count;           // 记录连通分量的数量
10
11     // 初始化: n 为节点数量
12     DSU(ll n) : parent(n + 1), siz(n + 1, 1), count(n) {
13         // 初始时每个节点的父节点是自己
14         std::iota(parent.begin(), parent.end(), 0);
15     }

```

```

16
17 // 查找 (Find) - 路径压缩
18 ll find(ll x) {
19     // 如果 x 不是根节点, 递归找根, 并进行路径压缩
20     return parent[x] == x ? x : parent[x] = find(parent[x]);
21 }
22
23 // 合并 (Union) - 按大小合并
24 // 返回值: true 表示合并成功 (原本不在一组), false 表示原本就在一组
25 bool merge(ll x, ll y) {
26     ll rootX = find(x);
27     ll rootY = find(y);
28
29     if (rootX == rootY) return false; // 已经在同一个集合
30
31     // 启发式合并: 把小的集合合并到大的集合上, 保持树的高度较低
32     if (siz[rootX] < siz[rootY]) std::swap(rootX, rootY);
33
34     parent[rootY] = rootX; // Y 挂到 X 上
35     siz[rootX] += siz[rootY]; // 更新 X 的大小
36     count--; // 连通分量减少一个
37     return true;
38 }
39
40 // 判断是否连通
41 bool connected(ll x, ll y) {
42     return find(x) == find(y);
43 }
44
45 // 获取某个节点所在集合的大小
46 int getSize(ll x) {
47     return siz[find(x)];
48 }
49 };

```

带删除并查集

基础并查集基于树形结构, 一旦某个节点被合并, 它可能成为很多其他节点的父节点。如果直接删除它或把它移到别的集合, 整棵树就断开了。

核心思想: 虚拟节点 (映射法)

既然真实的节点不能乱动, 我们就给每一个真实节点分配一个“虚拟信箱” (虚拟节点)。

- 合并操作: 我们实际上是在合并“虚拟节点”。
- 删除操作: 当我们要删除节点 x 时, 我们不改动原本树中的结构, 而是直接给节点 x 分配一个新的、独立的虚拟节点。
- 原来的节点就变成了一个没人用的“空壳” (僵尸节点), 这不会影响其他依赖它的子节点。

适用场景:

- 需要将某个节点从当前连通块中分离出来 (删除)。
- 需要将某个节点从集合 A 移动到集合 B。
- 动态图连通性问题中, 涉及节点失效的场景。

```

1 /**
2  * 带删除的并查集
3  * 核心原理: 使用 id 数组将真实节点映射到虚拟节点, 删除即重新映射。

```

```

4  */
5  struct DeletableDSU {
6      std::vector<int> parent;
7      std::vector<int> siz;
8      std::vector<int> id; // id[x] 表示真实节点 x 当前对应的虚拟节点编号
9      int virtual_cnt;    // 虚拟节点分配计数器
10
11     // 初始化: n 为初始节点数, max_ops 为预估的最大删除/移动操作次数
12     DeletableDSU(int n, int max_ops) :
13         parent(n + max_ops + 1),
14         siz(n + max_ops + 1, 1),
15         id(n + 1)
16     {
17         virtual_cnt = n;
18         // 初始时, 真实节点 1~n 映射到虚拟节点 1~n
19         for (int i = 1; i <= n; i++) {
20             id[i] = i;
21         }
22         // 初始化所有的虚拟节点
23         std::iota(parent.begin(), parent.end(), 0);
24     }
25
26     // 查找: 注意这里查找的是 id[x] 映射到的虚拟节点
27     int find(int x) {
28         int v = id[x]; // 先找到它当前的虚拟节点
29         return parent[v] == v ? v : parent[v] = find(parent[v]);
30     }
31
32     // 内部查找: 直接针对虚拟节点查找
33     int find_virtual(int v) {
34         return parent[v] == v ? v : parent[v] = find_virtual(parent[v]);
35     }
36
37     // 合并 x 和 y
38     bool merge(int x, int y) {
39         int rootX = find(x);
40         int rootY = find(y);
41         if (rootX == rootY) return false;
42
43         if (siz[rootX] < siz[rootY]) std::swap(rootX, rootY);
44
45         parent[rootY] = rootX;
46         siz[rootX] += siz[rootY];
47         return true;
48     }
49
50     // 删除/孤立 节点 x
51     void remove(int x) {
52         int rootX = find(x);
53         siz[rootX]--; // 原集合大小减一
54
55         // 给 x 分配一个新的独立虚拟节点
56         id[x] = ++virtual_cnt;
57         parent[id[x]] = id[x];
58         siz[id[x]] = 1;
59     }
60
61     // 将节点 x 移动到节点 y 所在的集合

```

```

62     void move(int x, int y) {
63         if (find(x) == find(y)) return;
64         remove(x); // 先把 x 独立出来
65         merge(x, y); // 再把 x 融进 y
66     }
67 };

```

带权并查集

基础并查集只能回答“ x 和 y 是不是一伙的”，但无法回答“在同一伙里， x 和 y 之间有什么具体关系”。

核心思想：边权维护（向量运算）

我们在节点指向父节点的边上增加一个权值 (weight)。

- 权值表示：weight[x] 记录的是节点 x 到其父节点的某种相对关系（比如距离差、分数差、胜负关系）。
- 路径压缩：在 find 时，不仅要把 x 挂到根节点上，还要把 weight[x] 更新为 x 到根节点的关系。
- 合并计算：在 merge 时，如果已知 x 到 y 的关系为 w ，通过向量加减法推导出根节点 $rootX$ 到 $rootY$ 的关系，从而连接两个树。

适用场景：

- 维护区间和（如：已知区间 $[l, r]$ 的和，问能否推断出另一区间的和）。
- 种类并查集/相对关系推断（如经典题：POJ 食物链、给出 $x - y = c$ 的多个方程判断是否冲突）。
- 在同一个连通块内，计算任意两点之间的差值/距离

```

1  /**
2   * 带权并查集
3   * 权值定义: weight[x] 表示节点 x 相对于其父节点的差值/距离
4   */
5  struct WeightedDSU {
6      std::vector<int> parent;
7      std::vector<ll> weight; // 记录到父节点的权值
8
9      WeightedDSU(int n) : parent(n + 1), weight(n + 1, 0) {
10         std::iota(parent.begin(), parent.end(), 0);
11     }
12
13     // 查找：路径压缩，同时更新权值
14     int find(int x) {
15         if (parent[x] == x) return x;
16
17         int old_parent = parent[x];
18         parent[x] = find(parent[x]); // 递归找根
19
20         // 核心: x 到新根的权值 = x 到旧父节点的权值 + 旧父节点到根的权值
21         weight[x] += weight[old_parent];
22
23         return parent[x];
24     }
25
26     // 合并: 已知 x 相对于 y 的权值为 w (例如: x - y = w)
27     // 规定方向: y 指向 x 的值为 w
28     bool merge(int x, int y, ll w) {
29         int rootX = find(x);
30         int rootY = find(y);

```

```

31
32         if (rootX == rootY) return false; // 已经在同一集合
33
34         // 将 rootX 挂在 rootY 下
35         parent[rootX] = rootY;
36
37         // 核心数学推导：向量加法
38         // x -> rootX (权值为 weight[x])
39         // y -> rootY (权值为 weight[y])
40         // 已知关系：x 相对于 y 是 w
41         // 所以 rootX 相对于 rootY 的权值应该如何推导？
42         // rootX_to_rootY = weight[y] - weight[x] + w;
43         weight[rootX] = weight[y] - weight[x] + w;
44
45         return true;
46     }
47
48     // 判断两个点是否在同一集合，并返回它们之间的权值差
49     // 假设查询 x 相对于 y 的差值
50     bool query(int x, int y, ll &result) {
51         if (find(x) != find(y)) {
52             return false; // 不连通，无法推断关系
53         }
54         // result = x_to_root - y_to_root
55         result = weight[x] - weight[y];
56         return true;
57     }
58 };

```

树状数组

树状数组是一种支持 **单点修改** 和 **区间查询** 的，代码量小的数据结构。

树状数组利用数的**二进制特征**来定义“管辖范围”，即一个数的二进制的最后一位 1 以及其后所有的 0 所构成的二进制大小，就是该数所管辖的区间范围。例如 8 的二进制为 1000，那么 a[8] 所管辖的范围就是 1-8 这个区间，再例如 7，a[7] 所管辖的范围就只有 7

lowbit(x) 用于提取 x 在二进制表示下最低位的 1 及其后面的 0 构成的数值。

```

1  struct BIT {
2      int n;
3      vector<ll> tree;
4      BIT(int n) : n(n), tree(n + 1, 0) {}
5
6      ll lowbit(int x){
7          return x & (-x);
8      }
9
10     // 在位置 i 增加 delta
11     void add(int i, ll delta) {
12         for (; i <= n; i += lowbit(i)) {
13             tree[i] = tree[i] + delta;
14         }
15     }
16
17     // 查询 1 到 i 的区间和
18     ll query(int i) {

```



```

19     ll sum = 0;
20     for (; i > 0; i -= lowbit(i)) {
21         sum = sum + tree[i];
22     }
23     return sum;
24 }
25
26 // 查询 l 到 r 的区间和
27 ll query_range(int l, int r) {
28     if (l > r) return 0;
29     return query(r) - query(l - 1);
30 }
31 };
32

```

线段树

区间修改+区间查询

```

1  ll arr[MAXN];          // 原数组
2  ll tree[MAXN * 4];     // 线段树数组
3  ll tag[MAXN * 4];      // 懒标记(加)
4  vector<ll> tag1(MAXN*4,1); // 懒标记(乘)
5
6  // 向上更新 (Push Up): 用子节点算父节点
7  void push_up(ll p) {
8      tree[p] = tree[p << 1] + tree[p << 1 | 1];
9  }
10
11 // 向下下放 (Push Down), 每当要访问数据的时候都要下放
12 // p: 当前节点, len: 当前节点管辖的区间长度
13 void push_down(ll p, ll len) {
14     // 如果存在修改乘上某值, 需要先对乘法的标记进行下放
15     if(tag1[p] != 1){
16         tag1[p<<1] *= tag1[p];
17         tag1[p<<1 | 1] *= tag1[p];
18
19         // 需要对加法的标记进行倍增
20         tag[p<<1] *= tag1[p];
21         tag[p<<1 | 1] *= tag1[p];
22
23         tree[p<<1] *= tag1[p];
24         tree[p<<1 | 1] *= tag1[p];
25
26         tag1[p] = 1;
27     }
28     if (tag[p]) {
29         // 1. 传给左子 (p<<1) 和 右子 (p<<1|1)
30         tag[p << 1] += tag[p];
31         tag[p << 1 | 1] += tag[p];
32
33         // 2. 更新子节点的值 (增量 = tag * 区间长度)
34         tree[p << 1] += tag[p] * (len - len / 2);
35         tree[p << 1 | 1] += tag[p] * (len / 2);
36

```

```

37         // 3. 清除当前标记
38         tag[p] = 0;
39     }
40 }
41
42 // 建树
43 void build(ll p, ll l, ll r) {
44     tag[p] = 0;
45     if (l == r) {
46         tree[p] = arr[l];
47         return;
48     }
49     ll mid = (l + r) >> 1;
50     build(p << 1, l, mid);
51     build(p << 1 | 1, mid + 1, r);
52     push_up(p);
53 }
54
55 // 区间修改: [ql, qr] 范围全加 k
56 void update(ll p, ll l, ll r, ll ql, ll qr, ll k) {
57     // 1. 完全覆盖, 直接更新并打标, 不递归
58     if (ql <= l && r <= qr) {
59         tree[p] += k * (r - l + 1);
60         tag[p] += k;
61         return;
62     }
63
64     // 2. 未完全覆盖, 先下放标记, 再递归
65     push_down(p, r - l + 1);
66     ll mid = (l + r) >> 1;
67     if (ql <= mid) update(p << 1, l, mid, ql, qr, k);
68     if (qr > mid) update(p << 1 | 1, mid + 1, r, ql, qr, k);
69     push_up(p);
70 }
71
72 void update1(ll p, ll l, ll r, ll ql, ll qr, ll k) {
73     if (ql <= l && r <= qr) {
74         tree[p] *= k;
75         tag1[p] *= k;
76         tag[p] *= k;
77         return;
78     }
79
80     push_down(p, r - l + 1);
81     ll mid = (l + r) >> 1;
82     if (ql <= mid) update(p << 1, l, mid, ql, qr, k);
83     if (qr > mid) update(p << 1 | 1, mid + 1, r, ql, qr, k);
84     push_up(p);
85 }
86
87 // 区间查询: 求 [ql, qr] 的和
88 ll query(ll p, ll l, ll r, ll ql, ll qr) {
89     if (ql <= l && r <= qr) return tree[p];
90
91     push_down(p, r - l + 1); // 查之前也要下放
92     ll mid = (l + r) >> 1;
93     ll res = 0;
94     if (ql <= mid) res += query(p << 1, l, mid, ql, qr);

```

```

95     if (qr > mid) res += query(p << 1 | 1, mid + 1, r, ql, qr);
96     return res;
97 }

```

区间最大值

只需在 `push_up` 中改为取 `max` 即可

珂朵莉树(odt)

它并不是一种传统的树形数据结构（如线段树、平衡树），而是一种基于 C++ STL 中的 `std::set` 来维护连续区间相同值的暴力数据结构/思想

1. 核心思想与适用场景

适用条件（极其苛刻但极其重要）：

1. 题目必须有**区间平推（区间推平/区间赋值）**操作，即将区间 $[L, R]$ 内的所有数字修改为同一个值 V 。
2. 题目的数据最好是**随机生成的**。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  using ll = long long;
4  #define endl '\n'
5
6  struct Node {
7      ll l, r;
8      mutable ll v;
9      bool operator<(const Node& o) const { return l < o.l; }
10 };
11
12 set<Node> odt;
13 ll n;
14
15 // 核心1: 分裂区间
16 auto split(ll x) {
17     if (x > n) return odt.end();
18     auto it = --odt.upper_bound({x, 0, 0});
19     if (it->l == x) return it;
20     ll l = it->l, r = it->r;
21     ll v = it->v;
22     odt.erase(it);
23     odt.insert({l, x - 1, v});
24     return odt.insert({x, r, v}).first;
25 }
26
27 // 核心2: 区间推平（灵魂所在，降维打击）
28 void assign(ll l, ll r, ll v) {
29     auto itr = split(r + 1);
30     auto itl = split(l);
31     odt.erase(itl, itr);
32     odt.insert({l, r, v});
33 }

```

```

34
35 // 应用1: 区间加法
36 void range_add(ll l, ll r, ll val) {
37     auto itr = split(r + 1);
38     auto itl = split(l);
39     for (auto it = itl; it != itr; ++it) {
40         it->v += val; // 因为有 mutable, 直接改
41     }
42 }
43
44 // 应用2: 区间求和
45 ll range_sum(ll l, ll r, ll x, ll mod) {
46     auto itr = split(r + 1);
47     auto itl = split(l);
48     ll res = 0;
49     for (auto it = itl; it != itr; ++it) {
50         res += (it->r - it->l + 1) * it->v;
51     }
52     return res;
53 }
54
55 // 扩展: 区间求第 K 小 (非常典型的 ODT 应用)
56 ll kth_smallest(ll l, ll r, ll k) {
57     auto itr = split(r + 1);
58     auto itl = split(l);
59     // 把区间抽出来排序
60     vector<pair<ll, ll>> vec;
61     for (auto it = itl; it != itr; ++it) {
62         vec.push_back({it->v, it->r - it->l + 1});
63     }
64     sort(vec.begin(), vec.end());
65     for (auto& p : vec) {
66         k -= p.second;
67         if (k <= 0) return p.first;
68     }
69 }
70
71 void solve()
72 {
73     // 初始化时, 把整个数组插入 ODT
74     // 假设初始数组全是 0
75     n = 100000;
76     odt.insert({1, n, 0});
77
78     // 如果初始数组有初值, 例如 a[1...n]
79     for(int i=1; i<=n; ++i) {
80         odt.insert({i, i, a[i]});
81     }
82 }
83
84 int main()
85 {
86     ios::sync_with_stdio(0);
87     cin.tie(0);
88     int t = 1;
89     // cin >> t;
90     while (t--)
91     {

```

```

92     solve();
93 }
94 return 0;
95 }

```

二分图

1.二分冲突模型

当题目中包含以下三个特征时

- **分成两组**：题目要求将一堆物品、人或节点，划分到**恰好两个**集合中（比如：分到 A 学院和 B 学院、分到两座监狱、染成黑白两色）。
- **两两关系**：任意两个元素之间存在某种“代价、反应值、冲突值”。
- **最值的最值**：
 - 题目的最终提问一定是以下两种句式之一：
 - 求同一组内的最大值，使其**尽可能小**。（最大值最小化）
 - **场景**：同一组内的人会打架，你要让同一组内打架最厉害的那对，伤害值尽可能小。
 - **二分目标X**：我们要让同组的最大冲突 $\leq X$;
 - **连边条件**：所有冲突值 $> X$ 的两人不能放在一起，所以连边后判断是否为二分图
 - 求同一组内的最小值，使其**尽可能大**。（最小值最大化）
 - **场景**：同一组内的人要合作，你要让同一组内最拉胯的那对，默契值尽可能大。
 - **二分目标X**：我们要让同组的最小默契 $\geq X$;
 - **连边条件**：所有默契值 $< X$ 的两人不能放在一起，所以连边后判断是否为二分图
- 模板：

```

1  vector<ll> vis(n+1,-1);
2  //判断是否为二分图
3  auto isg = [&](auto &&self,ll u,ll c,vector<vector<ll>> &vec) ->
bool{
4      vis[u] = c;
5      for(ll v : vec[u]){
6          if(vis[v]==c) return false;
7          if(vis[v] == -1 && !self(self,v,c^1,vec)) return false;
8      }
9      return true;
10 };
11 // 二分检查
12 auto check = [&](ll lim) -> bool{
13     vector<vector<ll>> vec(n+1);
14     for(auto [u,v,w] : adj){
15         if(w>lim){
16             vec[u].push_back(v);
17             vec[v].push_back(u);
18         }
19     }
20
21     vis.assign(n+1,-1);
22     for(int i = 1;i<=n;i++){
23         if(vis[i] == -1){
24             if (!isg(isg,i, 1, vec))
25                 return false;

```

```

26         }
27
28     }
29     return true;
30 };
31
32 int ans = 0;
33 int l = 0, r = a.size()-1; //所有值的个数，要记得去重
34 while(l<=r){
35     int mid = (l+r)>>1;
36     if(check(a[mid])){
37         // 依据情况逼近：
38         // 变体 A（求最小）：R = mid - 1;
39         // 变体 B（求最大）：L = mid + 1;
40         ans = a[mid];
41         // cout<<a[mid]<<" ";
42     }else{
43         //反向进行
44     }
45 }

```

ST表

ST 表 (Sparse Table, 稀疏表) 是一种非常经典且优雅的静态数据结构，专门用于解决**可重复贡献问题**（最常见的就是 **RMQ：区间最值查询**）。

它的核心优势在于极其变态的查询速度：只需一次 $O(N \log N)$ 的预处理，就能在 $O(1)$ 的时间复杂度内回答任意区间的查询。

一、核心思想：倍增 (Doubling) 与 DP

ST 表的本质是**动态规划 (DP)** *结合***倍增**思想。

普通的暴力查询是逐个遍历区间内的元素，而 ST 表通过预先计算好长度为 $2^0, 2^1, 2^2, \dots$ 的区间的答案，在查询时用两个长度为 2^k 的区间“拼凑”出目标区间。

1. 状态定义

我们设二维数组 $st[i][j]$ 表示：**从下标 i 开始，长度为 2^j 的区间内的最值**。

- 也就是区间 $[i, i + 2^j - 1]$ 的最值。
- 当 $j = 0$ 时，长度为 $2^0 = 1$ ，即 $st[i][0]$ 就是元素本身 $A[i]$ 。

2. 状态转移 (预处理)

如何求长度为 2^j 的区间的最值呢？

我们可以把它等分成两半，每半的长度正好是 2^{j-1} 。

- 前半的区间是 $[i, i + 2^{j-1} - 1]$ ，对应的状态是 $st[i][j-1]$ 。
- 后半的区间是从 $i + 2^{j-1}$ 开始，长度也是 2^{j-1} ，对应的状态是 $st[i + 2^{j-1}][j-1]$ 。

因此，转移方程（以求最小值为例）非常自然：

$$st[i][j] = \min(st[i][j-1], st[i + 2^{j-1}][j-1])$$

注意：在写代码时，外层循环必须是枚举区间长度（即 j ），内层循环枚举起点 i ，因为长区间的状态依赖于短区间的状态。

3. $O(1)$ 查询（拼凑区间）

假设我们要查询区间 $[L, R]$ 的最小值，区间长度为 $len = R - L + 1$ 。

我们找到一个最大的整数 k ，使得 $2^k \leq len$ （即 $k = \lfloor \log_2(len) \rfloor$ ）。

我们用两个长度为 2^k 的区间来覆盖目标区间 $[L, R]$ ：

- 从 L 往右、长度为 2^k 的区间：对应的状态是 $st[L][k]$ 。
- 从 R 往左、长度为 2^k 的区间：对应的状态是 $st[R - 2^k + 1][k]$ 。

这两个区间可能会重叠，但绝对不会超出 $[L, R]$ 的范围，并且完全覆盖了 $[L, R]$ 。

因为我们在求最值（min 或 max），一个元素被比较一次还是多次，不影响最终结果（这就是“可重复贡献问题”的性质）。

所以区间 $[L, R]$ 的最小值就是：

$\min(st[L][k], st[R - 2^k + 1][k])$

二、完整代码模板

以求解 **区间最小值** 为例（下标从 0 开始）：

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct SparseTable {
5      int n;
6      // st[j][i] 表示从 i 开始，长度为 2^j 的区间最小值
7      // 注意：把 j 放在第一维能提高缓存命中率 (Cache Friendly)，常数更小
8      vector<vector<int>> st;
9      vector<int> lg; // 预处理 log2 数组，实现真正的 O(1) 查询
10
11     void init(const vector<int>& a) {
12         n = a.size();
13         int max_log = __lg(n) + 1; // __lg(n) 是 GCC 内置函数，求向下取整的
log2(n)
14
15         st.assign(max_log, vector<int>(n));
16         lg.assign(n + 1, 0);
17
18         // 预处理 log 数组 (如果不用 __lg 可以用这个)
19         for (int i = 2; i <= n; i++) {
20             lg[i] = lg[i / 2] + 1;
21         }
22
23         // 初始化长度为 2^0 = 1 的区间
24         for (int i = 0; i < n; i++) {
25             st[0][i] = a[i];
26         }
27
28         // DP 预处理
29         for (int j = 1; j < max_log; j++) {
30             // i + (1 << j) <= n 保证右边界不越界
31             for (int i = 0; i + (1 << j) <= n; i++) {
32                 st[j][i] = min(st[j - 1][i], st[j - 1][i + (1 << (j - 1))]);
```

```

33         }
34     }
35 }
36
37 int query(int l, int r) {
38     if (l > r) swap(l, r);
39     int k = __lg(r - l + 1); // 也可以用预处理的 lg[r - l + 1]
40     // 两个区间求 min
41     return min(st[k][l], st[k][r - (1 << k) + 1]);
42 }
43 };

```

优势：

1. **查询极快**：纯正的 $O(1)$ ，在查询次数极其庞大（例如 10^6 或 10^7 级别）时，吊打线段树（ $O(\log N)$ ）。
2. **代码简短**：没有复杂的递归或树形结构维护，常数非常小。

劣势：

1. **必须是静态数组**：一旦初始化完成，**不支持任何修改操作**（单点修改、区间修改都不行）。如果需要修改，只能用线段树或树状数组。
2. **空间消耗较大**： $O(N \log N)$ 的空间在 $N = 10^7$ 时可能会导致内存超限（MLE）。
3. **只适用“可重复贡献问题”**：比如最值（max, min）、最大公约数（gcd）、按位与/或（&, |）。对于区间求和（ $\sum$ ），因为重叠部分会被加两次，所以不能用常规的 $O(1)$ 查询 ST 表（求和应使用前缀和或线段树）。

3.7 数学

一些定理？以及一些遇到的数学技巧

唯一分解定理

是指大于1的正整数n，都可以唯一地表示为有限个素数的乘积

$$n = p_1^{a_1} \times p_2^{a_2} \times \dots \times p_n^{a_n}$$

叉积求三角形面积

1. 利用向量的叉积

考虑空间坐标系 $Oxyz$, $\overrightarrow{OA} = (x_1, y_1, 0)$, $\overrightarrow{OB} = (x_2, y_2, 0)$, 则

$$S_{\triangle OAB} = \frac{1}{2} |\overrightarrow{OA}| \cdot |\overrightarrow{OB}| \cdot \sin \angle AOB = \frac{1}{2} |\overrightarrow{OA} \times \overrightarrow{OB}|$$

$$= \frac{1}{2} \left| \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ x_1 & y_1 & 0 \\ x_2 & y_2 & 0 \end{vmatrix} \right| = \frac{1}{2} \left| \begin{vmatrix} x_1 & y_1 \\ x_2 & y_2 \end{vmatrix} \mathbf{k} \right|$$

$$= \frac{1}{2} |x_1 y_2 - x_2 y_1|.$$

2. 推广

对于一般的 $\triangle ABC$, 设点 $A(x_1, y_1)$, $B(x_2, y_2)$, $C(x_3, y_3)$, 则

$$\begin{aligned}\overrightarrow{AB} &= (x_2 - x_1, y_2 - y_1), \\ \overrightarrow{AC} &= (x_3 - x_1, y_3 - y_1),\end{aligned}$$

所以

$$S_{\triangle ABC} = \frac{1}{2} \left| \det \begin{bmatrix} x_2 - x_1 & y_2 - y_1 \\ x_3 - x_1 & y_3 - y_1 \end{bmatrix} \right|.$$

裴蜀定理

设 a, b 是不全为零的整数. 那么, 对于任意整数 x, y , 都有 $\gcd(a, b) \mid ax + by$ 成立; 而且, 存在整数 x, y , 使得 $ax + by = \gcd(a, b)$ 成立.

推广:

设 $a_1, a_2, \dots, a_n$ 是不全为零的整数. 那么, 对于任意整数 $x_1, x_2, \dots, x_n$, 都有 $\gcd(a_1, a_2, \dots, a_n) \mid a_1x_1 + a_2x_2 + \dots + a_nx_n$ 成立; 而且, 存在整数 $x_1, x_2, \dots, x_n$, 使得 $\gcd(a_1, a_2, \dots, a_n) = a_1x_1 + a_2x_2 + \dots + a_nx_n$ 成立.

应用:

题意: 给出 n 张卡片并有一条无限长的纸带, 分别有 l_i 和 c_i , 你可以花 c_i 的钱来购买卡片然后选择在纸带上向前或向后跳 l_i 个单位任意次, 问你至少花多少钱才能走遍才能跳到纸带上的所有点

解: 分析该问题, 发现想要跳到每一个格子上, 必须使得所选数 $l_{i_1}, \dots, l_{i_k}$ 通过数次相加或相减得出的绝对值为 1. 也就是说, 存在整数 $x_1, \dots, x_k$ 使得 $l_{i_1}x_1 + \dots + l_{i_k}x_k = 1$. 由多个整数的裴蜀定理, 这相当于从数组 $l_1, \dots, l_n$ 中选择若干个数, 满足它们的最大公约数为 1, 同时要求代价和最小

如果每次只能向左或向右移动 a 步或 b 步, 能到达的所有位置一定是起点加上 $\gcd(a, b)$ 的倍数; 应用到 n 种可能的位移上也是一样的

枚举因数

```
1 vector<ll> a;  
2 for(ll i = 1; i * i <= n; i++){  
3     if(n % i == 0){  
4         a.push_back(i);          // 存入较小的因数  
5         if(i * i != n){  
6             a.push_back(n / i); // 存入对应的较大的因数 (注意是 n / i, 绝对不能写 n  
7             }  
8     }  
9 }
```

分解质因数

```
1  vector<ll> primes;
2  // 质因数分解, i 从 2 开始
3  for(ll i = 2; i * i <= n; i++){
4      // 只要能整除, 就一直除, 直到除不尽为止
5      while(n % i == 0){
6          primes.push_back(i);
7          n /= i; // 这里修改 n 是必须的! 因为我们要把因子剔除干净
8      }
9  }
10 // 如果最后 n 没被除到 1, 说明剩下一个大于 sqrt(原始n) 的质数
11 if(n > 1){
12     primes.push_back(n);
13 }
```

给定区间内求与n互质的数的个数

1. 核心思想：正难则反

要直接找“互质”的数很难，但找“不互质”的数很容易。

如果一个数 m 与 n 不互质，说明它们有**共同的质因数**。因此，我们只需要算出 $[1, X]$ 内有多少个数是 n 的质因数的倍数，然后用总数 X 减去这些数，剩下的就是互质的数。

2. 数学推导过程

假设 n 的所有**不同质因数**为 $p_1, p_2, \dots, p_k$ 。

- **步骤一：总数**

在 $[1, X]$ 范围内，整数的总个数是 X 。

- **步骤二：减去单个质因数的倍数**

$[1, X]$ 中，包含因子 p_i 的数有 $\lfloor X/p_i \rfloor$ 个。

我们要减去这些数。

- **步骤三：加回被重复减去的（两个质因数乘积的倍数）**

当我们减去 p_1 的倍数和 p_2 的倍数时，那些既是 p_1 又是 p_2 的倍数（也就是 $p_1 \times p_2$ 的倍数）被减了两次。为了平衡，我们需要**加回** $\lfloor X/(p_1 \times p_2) \rfloor$ 。

- **步骤四：减去加多了的（三个质因数乘积的倍数）**

同理，三个质因数乘积的倍数在前面“减、加”的过程中算错了，需要再次**减去** $\lfloor X/(p_1 \times p_2 \times p_3) \rfloor$ 。

以此类推，**奇数个质因数的乘积做减法，偶数个质因数的乘积做加法**。

```
1  //求[1,x]内与n互质的数的个数
2  ll count_coprime(ll x, const vector<ll>& p_factors) { //p_factors为n中的所有不同质因数
3      if (x == 0) return 0;
4      ll res = 0;
5      int k = p_factors.size();
```

```

6
7 // 用二进制枚举所有的组合
8 for (int mask = 0; mask < (1 << k); mask++) {
9     ll prod = 1;
10    int bits = 0; // 记录当前组合选了几个质因数
11
12    for (int i = 0; i < k; i++) {
13        if ((mask >> i) & 1) {
14            prod *= p_factors[i];
15            bits++;
16        }
17    }
18
19    // 奇数个质因数做减法，偶数个做加法
20    if (bits % 2 == 1) res -= x / prod;
21    else res += x / prod;
22 }
23 return res;
24 }

```

3.7.1 快速幂

快速幂是求解 a^b 的问题，其中 a, b 限定为整。如求3的 $1e18$ 次方，直接递推肯定超时。

原理：如求3的8次方，我们可以先算3的2次，再2次乘2次，到3的4次，4次乘4次，到3的8次

```

1  #include <iostream>
2  #include <bitset>
3  #include <cmath>
4  using namespace std;
5  using ll = long long;
6  ll qpow(ll a, ll b)
7  {
8      ll res = 1;
9      while (b != 0)
10     {
11         if (b & 1)
12             res *= a;
13         a *= a;
14         b /= 2;
15     }
16     return res;
17 }
18 int main()
19 {
20     ll a, b;
21     cin >> a >> b;
22     cout << qpow(a, b) << endl;
23     return 0;
24 }

```

3.7.2 矩阵快速幂

问题：快速求解 $n \times n$ 的矩阵A，求 A^b

```
1  #include <iostream>
2  #include <bitset>
3  #include <cmath>
4  #include <vector>
5  using namespace std;
6  using ll = long long;
7  const ll mod = 1e9 + 7;
8  vector<vector<ll>> mul(const vector<vector<ll>> &a, const vector<vector<ll>>
  &b) // 矩阵相乘
9  {
10     ll n = a.size() - 1;
11     vector<vector<ll>> res(n + 1, vector<ll>(n + 1, 0));
12     for (int i = 1; i <= n; ++i)
13     {
14         for (int j = 1; j <= n; ++j)
15         {
16             for (int k = 1; k <= n; ++k)
17             {
18                 res[i][j] = (res[i][j] + a[i][k] * b[k][j] % mod + mod) %
mod;
19             }
20         }
21     }
22     return res;
23 }
24 vector<vector<ll>> qpow(vector<vector<ll>> a, ll b) // 矩阵快速幂
25 {
26     ll n = a.size() - 1;
27     vector<vector<ll>> res(n + 1, vector<ll>(n + 1, 0));
28     for (int i = 1; i <= n; ++i)
29         res[i][i] = 1;
30     while (b)
31     {
32         if (b % 2)
33             res = mul(res, a);
34         a = mul(a, a);
35         b >>= 1;
36     }
37     return res;
38 }
39 int main()
40 {
41     ll n, b;
42     cin >> n >> b; // 矩阵阶数n, 次数b.
43     vector<vector<ll>> a(n + 1, vector<ll>(n + 1, 0));
44     for (int i = 1; i <= n; ++i)
45     {
46         for (int j = 1; j <= n; ++j)
47         {
48             cin >> a[i][j];
49         }
```

```

50     }
51     vector<vector<ll>> res = qpow(a, b);
52     for (int i = 1; i <= n; i++)
53     {
54         for (int j = 1; j <= n; ++j)
55         {
56             cout << res[i][j] << " ";
57         }
58         cout << endl;
59     }
60     return 0;
61 }

```

3.7.3 高精度

1. 加法高精度

```

1  //计算 123456789 + 987654321
2  /*
3  高精度的加法思想
4  1.把大数存到字符串;
5      2.字符串的每个字符数字都通过ASCII转换存到数组,
6      注意的是要低位存在数组开头:a[i] = s[len-i-1]-'0';
7
8      3.获取最大的数长度:max(len1,len2) ;
9      4.把a,b值加入到c数组: c[i] = a[i]+b[i];
10
11     5.c数组加法进位的算式:
12     ① c[i+1] += c[i]/10;
13     ② c[i] %= 10;
14
15     6.数字溢出,长度+1;
16     7.反向输出结果;
17 */
18 #include<iostream>
19 #include<string>
20 using namespace std;
21 string s1,s2;
22 int a[10000],b[10000],c[100001];
23 int main(){
24     // 1.输入值,长度
25     cin>>s1>>s2;
26     int len1 = s1.size();
27     int len2 = s2.size();
28     // 2.把字符转为整数存到数组
29     // 注意要个位存到数组开头
30     for(int i=0;i<len1;i++){
31         a[i] = s1[len1-i-1]-'0';
32     }
33     for(int i=0;i<len2;i++){
34         b[i] = s2[len2-i-1]-'0';
35     }
36     // 3.获取最大的数。
37     int len = max(len1,len2);
38     // 对各个位数进行相加

```

```

39     for(int i=0;i<len;i++){
40         c[i]=a[i]+b[i];
41     }
42     //4.进位
43     for(int i=0;i<len;i++){
44         c[i+1] += c[i]/10;
45         c[i] %= 10;
46     }
47     //5.溢出
48     while(c[len]==0 && len>0){
49         len--;
50     }
51     if(c[len]>0){
52         len++;
53     }
54     //6.反向输出
55     for(int i=len-1;i>=0;i--){
56         cout<<c[i];
57     }
58     return 0;
59 }

```

2. 高精度减法

```

1  // 辅助函数：判断 s1 是否小于 s2
2  bool isSmaller(const string& s1, const string& s2) {
3      if (s1.size() != s2.size()) {
4          return s1.size() < s2.size();
5      }
6      return s1 < s2; // 长度相同时直接用字典序比较
7  }
8
9  ll solve(string s1, string s2) {
10     // 1. 修复比较逻辑，不用 stoi
11     if (isSmaller(s1, s2)) {
12         swap(s1, s2);
13     }
14
15     // 2. 修复数组初始化，使用 vector 自动初始化为 0，防止垃圾值
16     // 开大一点防止溢出
17     int len1 = s1.size();
18     int len2 = s2.size();
19     vector<int> a(len1, 0);
20     vector<int> b(len1, 0); // 让 b 的大小和 a 一样，方便减法，不足补0
21     vector<int> c(len1, 0);
22
23     for (int i = 0; i < len1; i++) a[i] = s1[len1 - i - 1] - '0';
24     for (int i = 0; i < len2; i++) b[i] = s2[len2 - i - 1] - '0';
25
26     // 减法逻辑
27     for (int i = 0; i < len1; i++) {
28         if (a[i] < b[i]) {
29             a[i + 1]--;
30             a[i] += 10;
31         }

```

```

32     c[i] = a[i] - b[i];
33 }
34
35 // 去除前导零
36 int real_len = len1;
37 while (real_len > 1 && c[real_len - 1] == 0) {
38     real_len--;
39 }
40
41 // 3. 修复返回值: 直接在计算过程中取模, 而不是转成 stoi
42 // 结果现在存在 c[0]...c[real_len-1] 中, c[0] 是个位
43 ll num = 0;
44 // 从高位到低位还原数值并取模 (秦九韶算法)
45 for (int i = real_len - 1; i >= 0; i--) {
46     num = (num * 10 + c[i]) % MOD;
47 }
48
49 return num;
50 }

```

3. 高精度乘法

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4  const int MAXN = 40500; // 最大长度
5  int a[MAXN], b[MAXN], c[MAXN];
6  int main() {
7      string s1, s2;
8      cin >> s1 >> s2;
9      int n = s1.length(), m = s2.length(), len = n + m;
10     // 逆序存储
11     for (int i = 0; i < n; i++) a[n - i] = s1[i] - '0';
12     for (int i = 0; i < m; i++) b[m - i] = s2[i] - '0';
13     // 累加乘积
14     for (int i = 1; i <= n; i++) {
15         for (int j = 1; j <= m; j++) {
16             c[i + j - 1] += a[i] * b[j];
17         }
18     }
19     // 处理进位
20     for (int i = 1; i < len; i++) {
21         if (c[i] >= 10) {
22             c[i + 1] += c[i] / 10;
23             c[i] %= 10;
24         }
25     }
26     // 删除前导零并输出
27     while (len > 1 && c[len] == 0) len--;
28     for (int i = len; i > 0; i--) cout << c[i];
29     return 0;
30 }

```

3.7.4 离散化

1. **离散化差分** (Discretized Difference Array) 是解决“**大范围坐标、少操作次数**”区间问题的核心算法。

简单来说，就是当题目中告诉你坐标范围是 $1 \leq x \leq 10^9$ ，但操作次数只有 **N** 只有 10^5 时，因为内存和时间限制，你不能开一个 10^9 大小的数组。此时，我们需要把**用不到的中间空白坐标压缩掉**，或者**只存储有变化的坐标点**。

核心思想：

数轴是连续的，但数值的变化是**离散**的。

只有在区间的**起点和终点**，覆盖层数才会发生突变。在两个相邻的“关键点”之间，覆盖层数是恒定的。

因此，我们只需要记录这些**关键点（变化点）**，然后计算两个关键点之间的距离乘以当前的层数，就是这段区间的贡献。

```
1 map<ll, int> diff;
2 // 1. 读入并差分
3 for(int i=0; i<n; i++) {
4     cin >> l >> r;
5     diff[l]++;
6     diff[r+1]--; // 注意这里通常是 r+1, 代表左闭右开区间 [l, r]
7 }
8
9 // 2. 扫描线统计
10 ll ans = 0, sum = 0, pre = -1;
11 for(auto& [pos, val] : diff) {
12     // 第一次循环只记录起点, 不计算
13     if(pre != -1) {
14         // 计算上一段 [pre, pos) 的长度
15         ll len = pos - pre;
16         // 根据 sum (层数) 判断是否计入答案
17         if(sum > 0) ans += len;
18     }
19     sum += val; // 更新层数
20     pre = pos; // 更新上一个点
21 }
```

3.7.5 欧拉筛

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4 using ll = long long;
5 const ll maxn = 1e6;
6 vector<ll> prime; // 存储已经找到的所有素数 (相当于 “素数字典”);
7 vector<ll> phi(maxn + 1, 1); // 存储每个数的欧拉函数值 (顺带计算)
8 vector<bool> vis(maxn + 1, 0); // 标记某个数是否为合数 (false = 素数, true = 合数)
9 void init()
10 {
```



```

11     phi[1] = 1;
12     vis[1] = 1; // 1既不是质数也不是合数，但在筛法中标记为1避免重复处理
13     for (ll i = 2; i <= maxn; ++i)
14     {
15         if (!vis[i])
16         {
17             prime.push_back(i);
18             phi[i] = i - 1;
19         }
20         for (int j = 0; j < prime.size(); ++j)
21         {
22             ll p = prime[j];
23             if (p * i > maxn)
24                 break;
25             vis[p * i] = true;
26             if (i % p == 0)
27             {
28                 phi[i * p] = phi[i] * p;
29                 break;
30             }
31             else
32             {
33                 phi[i * p] = phi[i] * phi[p];
34             }
35         }
36     }
37 }
38 int main()
39 {
40     init();
41     for (int i = 1; i <= 10; i++)
42     {
43         cout << phi[i] << endl;
44     }
45     return 0;
46 }

```

3.7.6 逆元 取模 最大公约数(gcd) 最小公倍数(lcm)

取模最主要的是要注意步步取模，避免爆值，例如

- **加法:** $(a+b) \% \text{MOD}$
- **乘法:** $(a * b) \% \text{MOD}$
- **减法:** $(a - b + \text{MOD}) \% \text{MOD}$ (注意减法要先加上模数后再取模，避免出现负值取模)
- **除法:** 除法不能直接取模，需要求模逆元；除以一个数 b 等于乘以 b 的模逆元

$\frac{a}{b} (\text{mod } P) \equiv a * b^{P-2} (\text{mod } P)$ 可以用快速幂计算 `qpow(b, MOD-2)` (模数得是质数才成立，但一般题目给出的都是质数)

1到1e6模1e9+7的逆元模板

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4  using ll = long long;
5  const ll mod = 1e9 + 7;
6  const ll maxn = 1e6;
7  vector<ll> inv(maxn + 1, 1);
8  void init()
9  {
10     for (int i = 2; i <= maxn; ++i)
11     {
12         inv[i] = (mod - mod / i * inv[mod % i] % mod) % mod;
13     }
14 }
15 int main()
16 {
17     init();
18     for (int i = 1; i <= 10; ++i)
19     {
20         cout << inv[i] << endl;
21     }
22     return 0;
23 }
```

```
1  ll gcd(ll a, ll b)
2  {
3      a = abs(a);
4      b = abs(b);
5      if (a > b)
6          swap(a, b);
7      if (a == 0)
8          return b;
9      else
10         return gcd(b % a, a);
11 }
12 ll lcm(ll a, ll b)
13 {
14     return a / gcd(a, b) * b; // 先除后乘，尽量不溢
15 }
```

3.7.7排列数和组合数

```
1  vector<ll> fact(n+1), inv(n+1); //阶乘和逆元
2
3  ll qpow(ll a, ll b)
4  {
5      ll res = 1;
6      while (b != 0)
7      {
```

```

8         if (b % 2 == 1)
9             res = res * a % MOD;
10        a = a * a % MOD;
11        b /= 2;
12    }
13    return res;
14 }
15
16 void init(){
17     fact[0] = 1;
18     for(int i = 1; i <= n; i++){
19         fact[i] = (fact[i-1] * i) % MOD;
20     }
21     inv[n] = qpow(fact[n], MOD-2);
22     for(int i = n - 1; i >= 0; i--){
23         inv[i] = inv[i+1] * (i+1) % MOD;
24     }
25 }
26
27 ll c(ll n, ll k){ //n中选k
28     if(k < 0 || k > n) return 0;
29     return fact[n] * inv[k] % MOD * inv[n-k] % MOD;
30 }
31
32 ll A(ll n, ll k){
33     if(k < 0 || k > n) return 0;
34     return fact[n] * inv[n-k] % MOD;
35 }

```

3.8 位运算

```

1  __builtin_ctz(x) //获取x二进制后缀0的长度
2  __builtin_clz(x) //获取x二进制前缀0的长度 //二者皆为接受int类型，接受ll需要后面加ll
   //_
3
4  A + B = A^B + 2 * (A&B);
5  A | B = (A ^ B) ^ (A & B);

```

1. 线性基

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  using ll = long long;
4
5  struct LinearBasis {
6      ll d[64]; // 存储线性基的数组，d[i] 表示最高位为第 i 位的基向量
7      bool has_zero; // 记录原序列中是否能异或出 0（即是否存在线性相关的元素）
8
9      // 构造函数，初始化
10     LinearBasis() {
11         memset(d, 0, sizeof(d));

```

```

12     has_zero = false;
13 }
14
15 // 1. 插入一个元素 x
16 void insert(ll x) {
17     for (int i = 62; i >= 0; i--) { // 从最高位向下枚举 (long long 最大 62
位)
18         if ((x >> i) & 1) {           // 如果 x 的第 i 位是 1
19             if (!d[i]) {               // 如果这一位还没有基向量
20                 d[i] = x;               // x 成为这一位的基向量
21                 return;                 // 插入成功, 立刻退出
22             }
23             x ^= d[i];                  // 如果这一位已经被占了, 就用 d[i] 把 x
的第 i 位消成 0
24         }
25     }
26     // 如果 x 被消成了 0, 说明 x 可以由之前的基向量异或得到 (线性相关)
27     has_zero = true;
28 }
29
30 // 2. 查询原数组能异或出的最大值
31 ll query_max() {
32     ll res = 0;
33     for (int i = 62; i >= 0; i--) {
34         // 贪心策略: 如果异或上 d[i] 能让结果变大, 就异或它
35         if ((res ^ d[i]) > res) {
36             res ^= d[i];
37         }
38     }
39     return res;
40 }
41
42 // 3. 判断 x 能否由原数组的元素异或得到
43 bool check(ll x) {
44     for (int i = 62; i >= 0; i--) {
45         if ((x >> i) & 1) {
46             if (!d[i]) return false; // x 在第 i 位是 1, 但线性基没有这一位的
基, 无法消去
47             x ^= d[i];                // 消去第 i 位
48         }
49     }
50     return x == 0; // 如果 x 最终被消成 0, 说明它可以由线性基表出
51 }
52 };

```

对于线性基中的第 K 小与第 K 大问题, 核心在于利用消元重构后的独立性, 把“组合问题”转化为“二进制按位选择问题”。

一、先决条件：消元重构 (Simplify/Rebuild)

在查询之前, 必须先将线性基化为**简化阶梯型** (即前文提到的 `rebuild()`)。

重构的目的是让基元素之间**相互独立**:

- 设重构并提取出的非零基元素从小到大排列为 $d[0], d[1], d[2], \dots, d[m-1]$ (共 m 个)。
- 此时 $d[i]$ 的最高有效位 (第 i 个主位) 上为 1, 且其他更高的 $d[j]$ ($j > i$) 在这个主位上**全为 0**。

- **核心结论**：这 m 个元素能组合出 2^m 个互不相同的非负整数，并且**这些整数的大小顺序，完全由选取时用到的 d 的下标对应的二进制位决定。**

二、求第 K 小 (K-th Smallest)

求第 K 小时，我们需要注意**是否能组合出 0**：

- 如果原数组元素个数 $N > m$ ，说明原数组存在线性相关，可以组合出 0，此时 **0 是第 1 小的数**。
- 如果 $N = m$ ，则非空子集无法组合出 0。

算法步骤：

1. **边界判定**：可生成的不同数值总数为 $total = 2^m$ （若包含 0）或 $2^m - 1$ （若不含 0）。若 $K > total$ ，无解（返回 -1）。
2. **偏移修正**：如果可以组合出 0：
 - $K = 1$ 时直接返回 0；
 - $K > 1$ 时，由于 0 占用了第 1 小的位置，在非零组合中我们要找的是**第 $K - 1$ 小**，因此令 $K = K - 1$ 。
3. **二进制拆分**：将 K 转化为二进制。若 K 的第 i 位为 1，则结果异或上 $d[i]$ 。

三、求第 K 大 (K-th Largest)

“第 K 大”可以非常自然地**转化为求“第几小”**。

转化逻辑：

假设线性基可以生成 $Total$ 个不同的数（从 1 到 $Total$ 排名）：

- 最大的数（第 1 大）等于 **第 $Total$ 小**。
- 第 2 大的数等于 **第 $Total - 1$ 小**。
- **第 K 大的数等于 第 $Total - K + 1$ 小**。

算法步骤：

1. 计算总共能生成的不同数字个数 $Total$ ：
 - 可组合出 0 时， $Total = 2^m$ ；
 - 不可组合出 0 时， $Total = 2^m - 1$ 。
2. 若 $K > Total$ ，无解（返回 -1）。
3. 转化为求第 $K' = Total - K + 1$ 小，直接调用第 K 小的逻辑即可。

```
1  #include <iostream>
2  #include <vector>
3  #include <cstring>
4
5  using namespace std;
6
7  struct LinearBasis {
8      static const int MAX_BIT = 60; // 根据题目数据范围调整（例如 10^18 对应 60）
9      long long p[MAX_BIT + 1];      // 存储原始线性基
10     vector<long long> d;             // 存储重构后的线性基
11     int cnt;                         // 线性基中有效元素的个数（非零基底数）
12     int n;                           // 尝试插入的总元素个数
13
14     // 初始化
15     LinearBasis() {
```

```

16     memset(p, 0, sizeof(p));
17     cnt = 0;
18     n = 0;
19 }
20
21 // 1. 插入一个数
22 // 返回值: true 表示成功插入(线性无关), false 表示无法插入(线性相关)
23 bool insert(long long x) {
24     n++; // 记录插入的总数
25     for (int i = MAX_BIT; i >= 0; --i) {
26         if ((x >> i) & 1) {
27             if (!p[i]) {
28                 p[i] = x;
29                 cnt++;
30                 return true;
31             }
32             x ^= p[i];
33         }
34     }
35     return false;
36 }
37
38 // 2. 查询能异或出的最大值
39 long long query_max() {
40     long long res = 0;
41     for (int i = MAX_BIT; i >= 0; --i) {
42         if ((res ^ p[i]) > res) {
43             res ^= p[i];
44         }
45     }
46     return res;
47 }
48
49 // 3. 查询能异或出的最小非零值
50 long long query_min() {
51     for (int i = 0; i <= MAX_BIT; ++i) {
52         if (p[i]) return p[i];
53     }
54     return 0; // 只有线性基为空时才会返回 0
55 }
56
57 // 4. 重构线性基 (化为简化阶梯型, 用于第 K 大/小查询)
58 // 注意: 调用第 K 大/小查询前, 必须先调用一次此函数!
59 void rebuild() {
60     d.clear();
61     for (int i = MAX_BIT; i >= 0; --i) {
62         if (p[i]) {
63             for (int j = i - 1; j >= 0; --j) {
64                 if ((p[i] >> j) & 1) {
65                     p[i] ^= p[j];
66                 }
67             }
68         }
69     }
70     // 将重构后的非零元素按从小到大提取出来
71     for (int i = 0; i <= MAX_BIT; ++i) {
72         if (p[i]) {
73             d.push_back(p[i]);

```

```

74         }
75     }
76 }
77
78 // 5. 查询能异或出的第 k 小的值
79 // 返回 -1 表示 k 超出了能组合出的总数范围
80 long long query_kth_smallest(long long k) {
81     bool can_be_zero = (n > cnt); // 插入总数 > 基底数, 说明存在线性相关, 能
    组合出 0
82     long long total_combinations = (1LL << d.size()) - (can_be_zero ? 0
: 1);
83
84     if (k > total_combinations) return -1;
85
86     // 如果能组合出 0, 那么 0 就是第 1 小
87     if (can_be_zero) {
88         if (k == 1) return 0;
89         k--; // 扣除 0 占用的第 1 小位置
90     }
91
92     long long ans = 0;
93     for (int i = 0; i < d.size(); ++i) {
94         if ((k >> i) & 1) {
95             ans ^= d[i];
96         }
97     }
98     return ans;
99 }
100
101 // 6. 查询能异或出的第 k 大的值
102 // 返回 -1 表示 k 超出了能组合出的总数范围
103 long long query_kth_largest(long long k) {
104     bool can_be_zero = (n > cnt);
105     long long total_combinations = (1LL << d.size()) - (can_be_zero ? 0
: 1);
106
107     if (k > total_combinations) return -1;
108
109     // 第 k 大 等价于 第 (Total - k + 1) 小
110     long long k_small = total_combinations - k + 1;
111     return query_kth_smallest(k_small);
112 }
113 };

```

3.9 dp

1 树上dp

一、树形 DP 的本质是什么？

树形 DP 的本质是：利用树的“天然递归结构”，将大问题拆解为子树的小问题。

在一棵无根树中，只要我们人为规定一个节点为根（通常是 1 号点），它就变成了一棵有向的、层级分明的树。

- **无后效性**：一旦子树 v 的状态算好了，它内部怎么组合的就不再重要了，只会作为一个整体的数值提供给父亲。
- **最优子结构**：父亲 u 的最优解，一定可以由儿子 v 的最优解组合推导出来

标准遍历方向：自底向上（后序遍历）

即：先递归把所有儿子的 DP 值算出来 → 然后合并给父亲。

1.1 染色问题

大致题意：给你一颗树，树上可染红蓝绿三种颜色，且相邻节点不能同色，若一个节点有两个子节点，这两个子节点也不能同色，问你其中一个颜色可染节点的最大数量是多少

一般的解法是树上dp

```

1 //dp[u][0] 表示u节点不染要求颜色，1则为染
2 auto dfs = [&](auto &&self, ll u, ll fa) -> void{
3     ll sum = 0; //记录当前节点的子节点有多少已染对应颜色的点
4     ll maxx = 0;
5     for(auto v : adj[u]){
6         if(v != fa){
7
8             self(self, v, u);
9             sum += dp[v][0];
10            //求出若有一个子节点要染可产生的最大贡献
11            maxx = max(maxx, dp[v][1] - dp[v][0]);
12        }
13    }
14    }
15    //处理完所有子节点后，进行向上转移
16    dp[u][1] = 1 + sum; //如果u要染，则加上所有子节点不染所产生的最大贡献再加一
17    dp[u][0] = sum + maxx; //如果u不染，则加上所有子节点不染产生的最大贡献后再加上
    若其中一个子节点
18
19 };
20 dfs(dfs, 1, 0);
21 cout << max(dp[1][1], dp[1][0]) << endl;

```

1.2 树形背包

特征：给了一定的总额度（体积），让你在树上分配，通常要求选儿子必须先选父亲。

经典题目：P2014 [CTSC1997] 选课、二叉苹果树。

状态设计：dp[u][i] 表示以 u 为根的子树，分配了 i 个体积的最优解。

转移方程：嵌套循环，像极了分组背包。

```

1 // 在合并 v 到 u 时：
2 for(int j = M; j >= 0; j--) { // u 的总容量
3     for(int k = 0; k < j; k++) { // 分给 v 的容量
4         dp[u][j] = max(dp[u][j], dp[u][j-k] + dp[v][k]);
5     }
6 }

```


1.3 换根dp

特征：题目不问你以 1 为根的答案，而是问你：**以 i 为根时答案是多少**（要求输出所有 $i=1\dots N$ 的答案）。如果做 N 次 DFS 会超时。

经典题目：STA-Station、Tree Distances。

解法：两遍 DFS。

- **第一遍 DFS（自底向上）：**随便选一个根（比如 1），求出每个节点的 $dp[u]$ （仅考虑其子树内部的贡献）。
- **第二遍 DFS（自顶向下）：**开一个新数组 $ans[u]$ 。父亲 u 把自己身上除去 v 之外的其他部分，当作一个巨大的“上面挂着的子树”，下放给 v 。

[P3478 [POI 2008](#)] STA-Station - 洛谷

题意：让你求出每个节点为根时的所有节点的深度之和

```
1 void solve()
2 {
3     ll n;
4     cin>>n;
5     vector<vector<ll>> adj(n+1);
6     for(int i= 1;i<=n-1;i++){
7         ll u,v;
8         cin>>u>>v;
9         adj[u].push_back(v);
10        adj[v].push_back(u);
11    }
12
13    vector<ll> dept(n+1,0);
14    vector<ll> sz(n+1,0);
15    vector<ll> p(n+1,0);
16    auto dfs1 = [&](auto &&self,ll u,ll fa,ll d) ->void{
17        dept[u] = d;
18        sz[u] = 1;
19        for(ll v : adj[u]){
20            if(v != fa){
21                self(self,v,u,d+1);
22                sz[u]+=sz[v];
23            }
24        }
25    };
26    //第一次dfs求出所有节点的相对于1节点深度和子树大小
27    dfs1(dfs1,1,0,1);
28
29    for(int i = 1;i<=n;i++){
30        p[1]+=dept[i];
31    }
32    auto dfs2 = [&](auto &&self,ll u,ll fa) ->void{
33        for(ll v : adj[u]){
34            if(v != fa){
35                p[v] = p[u] - sz[v] + n - sz[v];
36                self(self,v,u);
37            }
38        }
39    };
40    //第二次求答案，思考容易得出从u转移到v的方程
```

```

41     dfs2(dfs2,1,0);
42
43     ll ans = -1;
44     ll id = -1;
45     for(int i = 1;i<=n;i++){
46         if(p[i]>ans){
47             ans = p[i];
48             id = i;
49         }
50     }
51     cout<<id<<endl;
52 }

```

2.基础dp

最长公共子序列(LCS)

```

1  int dp[5010][3010];
2  string s, t;
3  cin >> s >> t;
4
5  for (int i = 1; i <= s.length(); i++)
6  {
7      for (int j = 1; j <= t.length(); j++)
8      {
9          if (s[i-1] == t[j-1])
10         {
11             dp[i][j] = dp[i - 1][j - 1] + 1;
12         }
13         else
14         {
15             dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
16         }
17     }
18 }
19 cout<<dp[s.length()][t.length()]; //输出最长公共子序列长度
20
21 //反向追踪出最长的公共子序列
22 string res = "";
23 int i = s.length(),j = t.length();
24 while(i>0 && j>0){
25     if(s[i - 1] == t[j-1] ){
26         res+=s[i-1];
27         i--,j--;
28     }else{
29         if(dp[i-1][j] >= dp[i][j-1]) i--;
30         else j--;
31     }
32 }
33 cout<<res;
34

```

最长上升子序列(LIS)

设计 dp_i 为以 a_i 为结尾的最长上升子序列，计算时，尝试将 a_i 接到之前的最长不上降子序列后面

```
1  dp[1] = 1;
2  ll ans = 1;
3  for(int i = 2; i <= n; i++){
4      dp[i] = 1;
5      for(int j = 1; j < i; j++){
6          if(a[j] < a[i]){
7              dp[i] = max(dp[i], dp[j] + 1);
8              ans = max(ans, 1LL * dp[i]);
9          }
10     }
11 }
12 cout << ans;
```

n^2 的算法如果对 $1e5$ 及以上的数据来说有点慢，我们可以进行优化，可优化为 $O(n \log n)$ ，如下

```
1  vector<int> low(n + 1, 0);
2  int len = 0;
3  for (int i = 1; i <= n; i++)
4  {
5      if (b[i] > low[len]) //如果是最长不上降就是>=,也就是允许相等的情况
6      {
7          len++;
8          low[len] = b[i];
9      }
10     else
11     {
12         int idx = lower_bound(low.begin() + 1, low.begin() + len + 1, b[i]) -
13         low.begin();
14         low[idx] = b[i];
15     }
16 }
17 cout << len;
```

解释: low_i 表示长度为 i 的最长上升子序列；我们从1开始遍历到 n ，如果遇到当前数组的数值大于 low 中最后的元素，我们就可以把当前的数值接到后面；如果遇到严格小于 low 中最后的元素，我们就可以将其替换到第一个大于他的位置上，可以证明，这是更优的，因为如果后面的值越小，就更容易接上更多的值

1. 求【最长上升子序列 (严格递增 $<$)】

- **数组状态:** 升序。
- **延长条件:** `if (x > dp[len])`
- **替换位置:** 找第一个 $\geq x$ 的数替换掉它。
- **使用函数:** `lower_bound(..., x)` (默认就是升序)

2. 求【最长不上降子序列 (允许相等 $\leq$)】

- **数组状态:** 升序。

- **延长条件:** `if (x >= dp[len])`
- **替换位置:** 找第一个 `>x` 的数替换掉它。
- **使用函数:** `upper_bound(..., x)`

3. 求【最长下降子序列 (严格递减 $>$)】

- **数组状态:** 降序。
- **延长条件:** `if (x < dp[len])`
- **替换位置:** 找第一个 `≤x` 的数替换掉它。
- **使用函数:** `lower_bound(..., x, greater<ll>())`

4. 求【最长不上升子序列 (允许相等 $\geq$)】

- **数组状态:** 降序。
- **延长条件:** `if (x <= dp[len])`
- **替换位置:** 找第一个 `<x` 的数替换掉它。
- **使用函数:** `upper_bound(..., x, greater<ll>())`

求 **LCS** 的问题部分也可转化为求 **LIS** 的问题，例如，如果对应两个数组中的元素范围相同且每个数只出现一次，我们就可以把其中一个数组当作基准来调整另一个数组中的元素；

更具体的说，如果现在给你两个数组 a, b ，他们都是 n 的排列，让你求出 a 和 b 的最长公共子序列，我们就可以以 a 为基准，对 b 进行映射，那么问题就等价于对映射后的 b 求最长上升子序列

如下

```

1  int n;
2      cin >> n;
3      vector<int> pos(n+1,0);
4      vector<int> b(n + 1, 0);
5      for (int i = 1; i <= n; i++)
6      {
7          int a;
8          cin>>a;
9          pos[a] = i;
10     }
11     for (int i = 1; i <= n; i++)
12     {
13         cin >> b[i];
14         b[i] = pos[b[i]];
15     }
16
17     vector<int> low(n + 1, 0);
18     int len = 0;
19     for (int i = 1; i <= n; i++)
20     {
21         if (b[i] > low[len])
22         {
23             len++;
24             low[len] = b[i];
25         }
26         else
27         {
28             int idx = lower_bound(low.begin()+1, low.begin()+len+1, b[i]) -
low.begin();
29             low[idx] = b[i];

```

```

30     }
31     }
32
33     cout << len;
34

```

为什么正确？因为我们强行把a映射成了一个递增序列，将映射关系应用到b后；我们不难想到公共子序列的本质就是要求元素在两个数组中出现的相对顺序一致，那么由于这层映射关系，a单调递增了，那我们只用找出映射后的b中最长的单调递增序列即可；还可以知道，b中只要是单调递增的序列，那么这一段序列一定是a的一个合法子序列

3.11 杂项

莫队

莫队基础

莫队算法本质上是一种**极其优雅的暴力**，其本质是**分块**。它用于解决**离线区间查询**问题（“离线”意味着我们可以一次性读入所有查询，打乱顺序计算后再按原顺序输出）

如果已知区间 $[L, R]$ 的答案，能够快速的得到其相邻区间的答案，即可使用莫队算法来求解

莫队算法的精髓就在于：**通过对查询区间进行巧妙的排序，严格限制双指针移动的步数总和，将其优化到 $O(N\sqrt{N})$** ，为了做到这一点，我们需要分块，也就是把长度为 N 的数组分为 $\sqrt{N}$ 的块，每个块的长度为 $\sqrt{N}$ 。

1. 常规排序（基础版）

- **第一关键字：**左端点 L 所在的**块编号**从小到大排序。
- **第二关键字：**右端点 R 的位置从小到大排序。

2. 奇偶排序

- **第一关键字：**左端点 L 所在的块编号从小到大排序。
- **第二关键字：**如果 L 所在的块编号是**奇数**，则 R **从小到大**排序；如果 L 所在的块编号是**偶数**，则 R **从大到小**排序。

在莫队中，指针移动代码如下：

```

1 while (l > q.l) add(--l); // L左移，区间变大
2 while (r < q.r) add(++r); // R右移，区间变大
3 while (l < q.l) del(l++); // L右移，区间变小
4 while (r > q.r) del(r--); // R左移，区间变小

```

模板示例(以查询区间不同数字的个数为例)

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4
5 const int N = 200005; // 根据题目修改范围
6 int a[N];             // 原数组
7 int cnt[N];           // 记录元素的出现次数
8 int ans[N];           // 记录每个查询的最终答案

```

```

9  int current_ans; // 维护当前窗口内的答案
10 int block;      // 分块大小
11
12 // 1. 定义查询结构体
13 struct Query {
14     int l, r, id;
15 };
16 vector<Query> q;
17
18 // 2. 排序函数（奇偶排序，极力推荐！）
19 bool cmp_oddeven(const Query& x, const Query& y) {
20     if (x.l / block != y.l / block) {
21         return x.l / block < y.l / block; // 第一关键字：L所在的块
22     }
23     // 第二关键字：奇数块升序，偶数块降序
24     if ((x.l / block) & 1) return x.r < y.r;
25     else return x.r > y.r;
26 }
27
28 // （备用）常规排序（最容易懂的写法）
29 bool cmp_standard(const Query& x, const Query& y) {
30     if (x.l / block != y.l / block) {
31         return x.l / block < y.l / block;
32     }
33     return x.r < y.r;
34 }
35
36 // 3. 扩大区间时的状态转移（根据题目逻辑修改）
37 inline void add(int pos) {
38     if (cnt[a[pos]] == 0) {
39         current_ans++; // 如果加入前数量为0，说明出现了一个新数字
40     }
41     cnt[a[pos]]++;
42 }
43
44 // 4. 缩小区间时的状态转移（根据题目逻辑修改）
45 inline void del(int pos) {
46     cnt[a[pos]]--;
47     if (cnt[a[pos]] == 0) {
48         current_ans--; // 如果移出后数量为0，说明少了一种数字
49     }
50 }
51
52 void solve() {
53     int n, m;
54     if (!(cin >> n >> m)) return; // 应对多组输入
55
56     for (int i = 1; i <= n; i++) {
57         cin >> a[i];
58     }
59
60     // 设定块的大小，通常设定为 sqrt(N)
61     block = max(1.0, sqrt(n));
62     // 注：若 N 和 M 差距极大，理论最优块大小是 max(1.0, n / sqrt(m))
63
64     q.resize(m + 1);
65     for (int i = 1; i <= m; i++) {
66         cin >> q[i].l >> q[i].r;

```

```

67     q[i].id = i; // 记录原始查询顺序
68 }
69
70 // 将查询数组进行排序（下标从1开始到m结束）
71 sort(q.begin() + 1, q.begin() + 1 + m, cmp_oddeven);
72
73 // 5. 初始化双指针（极其重要：设置为一个空区间）
74 int l = 1, r = 0;
75 current_ans = 0;
76 // 注意：如果需要多组测试数据清空，还要 memset(cnt, 0, sizeof(cnt));
77
78 // 6. 执行莫队主循环
79 for (int i = 1; i <= m; i++) {
80     int ql = q[i].l;
81     int qr = q[i].r;
82     int id = q[i].id;
83
84     // 核心：先扩大（add），再缩小（del）
85     // 前缀自减、前缀自加、后缀自增、后缀自减千万别写错！
86     while (l > ql) add(--l);
87     while (r < qr) add(++r);
88     while (l < ql) del(l++);
89     while (r > qr) del(r--);
90
91     // 记录离线答案
92     ans[id] = current_ans;
93 }
94
95 // 7. 按原顺序输出答案
96 for (int i = 1; i <= m; i++) {
97     cout << ans[i] << '\n';
98 }
99 }
100
101 int main() {
102     ios::sync_with_stdio(false);
103     cin.tie(nullptr);
104     cout.tie(nullptr);
105
106     int t = 1;
107     // cin >> t;
108     while (t--) {
109         solve();
110     }
111     return 0;
112 }

```

双堆维护中位数

```

1 struct MedianFinder {
2     priority_queue<ll> maxp; // 大根堆，存较小的一半
   (含中位数)
3     priority_queue<ll, vector<ll>, greater<ll>> minp; // 小根堆，存较大的一半

```

```

4
5 unordered_map<ll,int> delMax, delMin; // 分别记录两个堆各自"待懒删除"的值及次
数
6 ll szMax = 0, szMin = 0; // 两个堆各自的真实（未被删除）元素个数
7
8 // 清理 maxp 堆顶已经被标记删除的元素
9 void cleanMax(){
10     while(!maxp.empty()){
11         auto it = delMax.find(maxp.top());
12         if(it == delMax.end() || it->second == 0) break;
13         it->second--;
14         maxp.pop();
15     }
16 }
17 // 清理 minp 堆顶已经被标记删除的元素
18 void cleanMin(){
19     while(!minp.empty()){
20         auto it = delMin.find(minp.top());
21         if(it == delMin.end() || it->second == 0) break;
22         it->second--;
23         minp.pop();
24     }
25 }
26
27 // 根据真实计数 szMax / szMin 重新调整平衡
28 void rebalance(){
29     while(szMax > szMin + 1){
30         cleanMax();
31         minp.push(maxp.top()); maxp.pop();
32         szMax--; szMin++;
33     }
34     while(szMin > szMax){
35         cleanMin();
36         maxp.push(minp.top()); minp.pop();
37         szMin--; szMax++;
38     }
39     cleanMax(); cleanMin();
40 }
41
42 // 插入一个数
43 void add(ll x){
44     cleanMax();
45     if(maxp.empty() || x <= maxp.top()){ maxp.push(x); szMax++; }
46     else { minp.push(x); szMin++; }
47     rebalance();
48 }
49
50 // 删除一个数（要求这个数确实存在于结构中，否则计数会出错）
51 void erase(ll x){
52     cleanMax(); cleanMin();
53     if(!maxp.empty() && x <= maxp.top()){
54         delMax[x]++; szMax--; // 逻辑上认为 x 属于"较小的一半"
55     } else {
56         delMin[x]++; szMin--; // 属于"较大的一半"
57     }
58     cleanMax(); cleanMin(); // 如果刚好在堆顶，立刻顺手清掉
59     rebalance();
60 }

```



```

61
62     ll size() const {
63         return szMax + szMin;
64     }
65
66     bool empty() const {
67         return size() == 0;
68     }
69
70     // 中位数为浮点数版本（总数为偶数时取两堆堆顶均值）
71     double getMedian(){
72         cleanMax(); cleanMin();
73         if(szMax > szMin) return (double)maxp.top();
74         return (maxp.top() + minp.top()) / 2.0;
75     }
76
77     // 中位数保证是整数时使用
78     ll getMedianInt(){
79         cleanMax(); cleanMin();
80         if(szMax > szMin) return maxp.top();
81         return (maxp.top() + minp.top()); // 需要 /2 的话自行调整
82     }
83
84     // 若元素个数为偶数，分别取两个中位数（下中位数、上中位数）
85     pair<ll,ll> getTwoMedians(){
86         cleanMax(); cleanMin();
87         // 调用前需保证 size() 为偶数且 size() > 0
88         return {maxp.top(), minp.top()};
89     }
90 };

```

4. 一些类型题的处理思路

4.1 存在大量插入，删除以及查询全局最小(大)值的

1. 方法一：使用multiset

- 适用场景：总操作次数(插入+删除)在 10^5 级别，且时间为 2 秒可用
- 插入：`ms.insert(x)`，删除一个元素：`ms.erase(ms.find(x))` 均为 $O(\log n)$

2. 方法二：双优先队列

- 适用场景：极高频的插入和删除，且只关心最大值或最小值
- 使用：定义两个最小堆的pq，一个存放所有插入的元素，一个存放所有删除的元素，查询时，如果两个堆顶的元素都相同，说明该值已被删除，对两个堆执行 *pop* 操作,直到不相同为止，该次查询的值就是负责存放的堆的堆顶元素

3. 方法三：动态开点线段树

5. 优化技巧

线段树优化建图

如果遇到 **点到区间** 连边或 **区间到点** 连边，且总边数和总操作数都很大的时候，且需要跑**最短路**等算法的时候一般都会使用这种技巧

例: [Problem - B - Codeforces](#)

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  using ll = long long;
4  #define endl '\n'
5  #define pll pair<ll,ll>
6  #define T tuple<ll,ll,ll>
7
8  const ll INF = 1e18;
9
10 struct Node{
11     ll to;
12     ll w;
13 };
14
15 void solve()
16 {
17     ll n,q,s;
18     cin>>n>>q>>s;
19     vector<vector<Node>> > adj(4*n);
20     vector<ll> in(4*n,0); //入树, 点到区间
21     vector<ll> out(4*n,0); //出树, 区间到点
22     ll cnt = n; //虚拟节点编号需大于所有真实点
23
24     //建树过程
25     auto build = [&](auto &&self,ll p,ll l,ll r) -> void{
26         if(l == r){
27             in[p] = 1;
28             out[p] = 1;
29             return;
30         }
31
32         ll ls = p<<1;
33         ll rs = p<<1 | 1;
34         ll mid = (l+r)>>1;
35
36         in[p] = ++cnt;
37         out[p] = ++cnt;
38
39         self(self,ls,l,mid);
40         self(self,rs,mid+1,r);
41
42         adj[in[p]].push_back({in[ls],0});
43         adj[in[p]].push_back({in[rs],0});
44
45         adj[out[ls]].push_back({out[p],0});
46         adj[out[rs]].push_back({out[p],0});
47     };
48
49     //点到区间连边
50     auto v_to_range = [&](auto &&self,ll p,ll v,ll w,ll l,ll r,ll ql,ll qr)
51     ->void{
52         if(ql<=l && r<=qr){
```

```

52         adj[v].push_back({in[p],w});
53         return;
54     }
55
56     ll ls = p << 1;
57     ll rs = p << 1 | 1;
58     ll mid = (l + r) >> 1;
59
60     if(ql<=mid) self(self,ls,v,w,l,mid,ql,qr);
61     if(qr>mid) self(self,rs,v,w,mid+1,r,ql,qr);
62 };
63
64     auto range_to_v = [&](auto &&self,ll p,ll v,ll w,ll l,ll r,ll ql,ll qr)
-> void{
65         if(ql<=l && r<=qr){
66             adj[out[p]].push_back({v,w});
67             return;
68         }
69
70         ll ls = p << 1;
71         ll rs = p << 1 | 1;
72         ll mid = (l + r) >> 1;
73
74         if(ql<=mid) self(self,ls,v,w,l,mid,ql,qr);
75         if(qr>mid) self(self,rs,v,w,mid+1,r,ql,qr);
76     };
77
78     build(build,1,1,n);
79
80     while(q--){
81         ll t;
82         cin>>t;
83         if(t==1){
84             ll v,u,w;
85             cin>>v>>u>>w;
86             adj[v].push_back({u,w});
87         }else{
88             ll v,l,r,w;
89             cin>>v>>l>>r>>w;
90             if(t==2){
91                 v_to_range(v_to_range,1,v,w,1,n,l,r);
92             }else if(t == 3){
93                 range_to_v(range_to_v,1,v,w,1,n,l,r);
94             }
95         }
96     }
97
98     vector<ll> dist(cnt+1,INF);
99     priority_queue<p11,vector<p11>,greater<p11>> pq;
100     pq.push({0,s});
101     dist[s] = 0;
102
103     while(!pq.empty()){
104         auto [d,u] = pq.top();
105         pq.pop();
106
107         if(d>dist[u]) continue;
108

```

```

109         for(auto &edge : adj[u]){
110             ll v = edge.to;
111             ll w = edge.w;
112             if(dist[v]>dist[u]+w){
113                 dist[v] = dist[u] + w;
114                 pq.push({dist[v],v});
115             }
116         }
117     }
118
119     for(ll i = 1;i<=n;i++){
120         if(dist[i] == INF){
121             cout<<-1<<" ";
122         }else{
123             cout<<dist[i]<<" ";
124         }
125     }
126
127 }
128
129 int main()
130 {
131     ios_base::sync_with_stdio(false);
132     cin.tie(NULL);
133     ll t = 1;
134     // cin >> t;
135
136     while (t--)
137     {
138         solve();
139     }
140     return 0;
141 }

```

调和级数

与其对数组b中每个数寻找数组a中是否存在能整除他的数，不如枚举数组a中的数的倍数后，看看b数组中是否有对应的值

因为前者的复杂度为 $O(n^2)$ 后者则是 $O(n \ln n)$

后者代码可大致表示如下

```

1  int n = 1000000;
2  for (int i = 1; i <= n; i++) {
3      for (int j = i; j <= n; j += i) { // 注意这里是 j += i，而不是 j++
4          // 执行某些 O(1) 的操作
5      }
6  }

```

总操作次数T为

$$T = \frac{n}{1} + \frac{n}{2} + \dots + \frac{n}{n}$$

$$T = n \times \left(\frac{1}{1} + \frac{1}{2} + \dots + \frac{1}{n} \right)$$

其中

$$\sum_{i=1}^n \frac{1}{i} \approx \ln(n)$$

所以时间复杂度是 $O(n \ln(n))$

值得注意的坑

`sqrt(n)`的浮点误差

用 `sqrt()` (浮点数) 直接当数组下标, 存在精度问题

`sqrt(r)` 返回的是 `double`, 对于完全平方数, 浮点误差可能导致:

- `sqrt(100000000)` 算出 `9999.99999999...`, 被截断成 `9999` 而不是 `10000`, 少算一个;

```
1  ll isqrt(ll x){
2      ll r = (ll)sqrtl((long double)x);
3      while (r > 0 && r*r > x) r--;
4      while ((r+1)*(r+1) <= x) r++;
5      return r;
6  }
```

小点

```
1  ceil(x) //取第一个不小于x的整数 <cmath>
```

```
1  //加速
2  ios::sync_with_stdio(false);
3  cin.tie(0);
4  cout.tie(0);
5
6  // 2. 去重 unique 会将重复的元素移到末尾, 返回去重后最后一个有效元素的下一个位置的迭代器
7  //必须先排序
8  auto last = unique(a.begin(), a.end());
9  a.erase(last, a.end()); //配合vector::erase s
10 a.erase(unique(a.begin(), a.end()), a.end()); /
11
12
13 //用endl再极端情况下可能会TLE, 建议直接使用'\n'
14 //或者直接 #define endl '\n'
15
16 //row+i 表示同一副对角线
17 //row-i 表示同一主对角线 因为会出现负值, 所以我们通常会加一个常数,
18 //row-i+n
19 cout << fixed << setprecision(8) << value << endl; // 改成8精度
```

```

20
21 //整数向上取整可以 (a+b-1)/b
22
23 对于矩阵中的任意2*2子块，想要其合为合数，只需顺序填入值
24
25
26
27 //对于一个矩阵有一点(r,c),顺时针旋转后为(c,len-1-r),逆时针旋转后为(len-1-c,r),len为
    矩阵边长
28 //当然这里的坐标是相对坐标并不是全局坐标，即左上角为(0,0)
29
30 勾股数的构造：给定一个数x，满足x^2+y^2=z^2;
31 if(x&1){
32     y = (x*x)/2;
33     z = (x*x+1)/2;
34 }else{
35     y = (x*x)/4-1;
36     z = (x*x)/4+1;
37 }
38
39 //读数据时内容中含有空格等制表符不能用cin，要用getline(cin,s);
40 //一个长度为n的字符串能产生n(n+1)/2个子串
41 //对于二维前缀和
42 pre[i][j] = pre[i-1][j] + pre[i][j-1] - pre[i-1][j-1] + vec[i][j];
43 //对于其局部，当左上角坐标为(i,j),右下角为(ii,jj)时
44 sum = pre[ii][jj] - pre[i-1][jj] - pre[ii][j-1] + pre[i-1][j-1];
45
46 在 1e7 的数据范围内，相邻质数的最大间隙只有 154

```

图论

1. 对于任何森林（没有环的图），连通块数量 C 满足 $c = \deg[u] - \deg[v] - 1 - is_adj(u, v)$
其中 $is_adj(u, v)$ 表示 u 和 v 是否为连边，是则为1，反之为0;
2. 对树上任意三个点 u, v, w ，一定存在唯一一个点 m （可能是 u, v, w 之一，也可能是中间的某个点），使得从 m 出发有三条互不相交的"手臂"分别通向 u, v, w 。这个 m 就叫**中位点**

$i \equiv j \pmod{x}$ 意味着 i 和 j 关于 x 同余，那么集合可表示为 $\{1 + x, 1 + 2x, \dots, 1 + kx\}$

0-base 下 $[L, R]$ 的和用前缀和表示为 $pre[R+1] - pre[L]$

1-base 下 $[L, R]$ 的和用前缀和表示为 $pre[R] - pre[L-1]$

只由0或1组成的二维矩阵，只要该点为1(或0)且1(或0)的总数大于等于2时，总能找到一个终点，使得路径上排成的字符为回文串

对于一组升序排序的数组，如果任意两个数之差小于等于某一个值，那么整个数组中的数据可以被两两配对，且之差不大于那个值

如果再进行二分查找的时候成员函数有，直接用成员函数，不要用std::的，例如multiset有upper_bound和lower_bound函数

好用函数

`std::is_sorted(a.begin(), a.end());` //判断数组是否是非降序的 既检查是否满足 $a_i \leq a_{i+1}$, 是返回true, 否则返回false

如果想检查是否为降序可添加 `std::greater<ll>()`

`std::is_sorted_until` 该函数返回一个迭代器, 返回到哪个元素开始不再是非降序

`std::binary_search`(起始迭代器, 结束迭代器, 要查找的值) 该函数返回一个bool值, 可查找数组中是否存在特点的值, 存在返回 *true* 否则 返回 *false*, 当然数组得是单调的 该函数复杂度为 $O(\log n)$

`std::transform()` 复杂度: $O(n)$, 因为底层由 `for` 循环实现

```
1 //第一种用法: 一元操作 (处理单个区间)
2 //这是最常用的情况: 将一个区间里的每个元素, 经过某种转换后, 存入目标区间。
3 //transform(起始迭代器1, 结束迭代器1, 目标起始迭代器, 一元操作函数);
4 transform(s.begin(), s.end(), s.begin(), ::toupper); //将字符串从开始到结尾的字符全都
   转换为大写, 再存回s; ::tolower, 可转换为小写, 注意都是::, 而并非是std::
5
6 //第二种用法: 二元操作, 可结合lambda使用
7 //transform(起始1, 结束1, 起始2, 目标起始, 二元操作函数);
8 vector<int> a = {1, 2, 3};
9 vector<int> b = {4, 5, 6};
10
11 // 提前分配好结果数组的空间!
12 vector<int> res(3);
13
14 // 将 a 和 b 对应位置相加, 结果存入 res
15 // std::plus<int>() 是标准库提供的一个仿函数, 等价于返回 x + y
16 transform(a.begin(), a.end(), b.begin(), res.begin(), plus<int>());
17 //transform(a.begin(), a.end(), b.begin(), res.begin(), [](int x,int y){
18 //    return x+y;
19 //});
20 //若不想提前分配空间, 可在目标起始填入 std::back_inserter(res)
21 for(auto x:res){
22     cout<<x<<" ";
23 }
```

自己的小壁画

在递归的过程中, 例如dfs, 在进行 `for(auto v : adj[u])` 的过程中, 在for内部是对u的孩子节点进行操作的, 而在for循环结束后则是对所有孩子节点给出的信息进行操作, 这点在树上dp的时候要注意